Rockchip RK3572 USB Developer Guide

ID: RK-SM-YF-C67

Release Version: V1.2.0

Release Date: 2026-06-22

Security Level: □Top-Secret □Secret □Internal ■Public

DISCLAIMER

THIS DOCUMENT IS PROVIDED “AS IS”. ROCKCHIP ELECTRONICS CO., LTD. (“ROCKCHIP”) DOES NOT PROVIDE ANY WARRANTY OF ANY KIND, EXPRESSED, IMPLIED OR OTHERWISE, WITH RESPECT TO THE ACCURACY, RELIABILITY, COMPLETENESS, MERCHANTABILITY, FITNESS FOR ANY PARTICULAR PURPOSE OR NON-INFRINGEMENT OF ANY REPRESENTATION, INFORMATION AND CONTENT IN THIS DOCUMENT. THIS DOCUMENT IS FOR REFERENCE ONLY. THIS DOCUMENT MAY BE UPDATED OR CHANGED WITHOUT ANY NOTICE AT ANY TIME DUE TO THE UPGRADES OF THE PRODUCT OR ANY OTHER REASONS.

Trademark Statement

"Rockchip", "瑞芯微", "瑞芯" shall be Rockchip’s registered trademarks and owned by Rockchip. All the other trademarks or registered trademarks mentioned in this document shall be owned by their respective owners.

All rights reserved. ©2026. Rockchip Electronics Co., Ltd.

Beyond the scope of fair use, neither any entity nor individual shall extract, copy, or distribute this document in any form in whole or in part without the written approval of Rockchip.

Rockchip Electronics Co., Ltd.

No.18 Building, A District, No.89, software Boulevard Fuzhou, Fujian,PRC

Website: www.rock-chips.com

Customer service Tel: +86-4007-700-590

Customer service Fax: +86-591-83951833

Customer service e-Mail: fae@rock-chips.com

Preface

Overview

This document provides a development guide for the RK3572 USB module, aimed at helping developers understand the hardware circuit design and software DTS configuration of the RK3572 USB controller and PHY, so that developers can flexibly design and develop quickly according to the USB application needs of the product.

Product Version

Chipset Models	Kernel Version
RK3572	Linux-6.12

Intended Audience

This document (this guide) is mainly applicable to the following engineers:

- Technical Support Engineers
- Software Development Engineers
- Hardware Development Engineers

Revision History

Date	Version	Author	Description
2026-04-22	V1.0.0	William Wu Frank Wang	Initial version
2026-05-13	V1.1.0	William Wu	Fix RK809 Type-C controller chip name
2026-06-22	V1.2.0	William Wu	Modified document format according to Rockchip documentation writing specifications

Contents

Rockchip RK3572 USB Developer Guide

- 1. Introduction to RK3572 USB Controllers and PHYs
- 2. RK3572 USB Hardware Circuit Design
  - 2.1 USB Controller Power Supply and Power Management
  - 2.2 USB PHY Power Supply and Power Management
    - 2.2.1 USB 2.0 PHY Power Supply and Power Management
    - 2.2.2 USB 3.0 PHY Power Supply and Power Management
  - 2.3 USB Hardware Circuit Design Considerations
    - 2.3.1 USB DRD0 VBUSDET Circuit Design
    - 2.3.2 Maskrom USB Circuit Design
    - 2.3.3 Type-C USB 3.0 Hardware Circuit
      - 2.3.3.1 Type-C USB 3.0 Switch Hardware Circuit Design
      - 2.3.3.2 Type-C USB 2.0 Switch Hardware Circuit Design
    - 2.3.4 Type-C USB 2.0 with PD Hardware Circuit
    - 2.3.5 Type-C USB 2.0 only Hardware Circuit
    - 2.3.6 Type-A USB 3.0 Hardware Circuit
    - 2.3.7 Multi Type-A USB 3.0 Hardware Circuit
- 3. RK3572 USB DTS Configuration
  - 3.1 USB Chip-Level DTSI Configuration
    - 3.1.1 USB Controller DTSI Configuration
    - 3.1.2 USB PHY DTSI Configuration
  - 3.2 Type-C USB 3.0 Switch Full-Function DTS Configuration
  - 3.3 Type-C USB 2.0 Switch Full-Function DTS Configuration
  - 3.4 Type-C USB 2.0 with PD DTS Configuration
  - 3.5 Type-C USB 2.0 only DTS Configuration
  - 3.6 Type-A USB 3.0 DTS Configuration
  - 3.7 Multi Type-A USB 3.0 DTS Configuration
  - 3.8 USB3.0 Combo PHY Unpowered DTS Configuration
    - 3.8.1 Combo PHY0 Unpowered DTS Configuration
    - 3.8.2 Combo PHY1 Unpowered DTS Configuration
  - 3.9 USB DTS Important Property Configuration Instructions
    - 3.9.1 USB Controller DTS Property Configuration Instructions
    - 3.9.2 USB 2.0 PHY DTS Property Configuration Instructions
    - 3.9.3 USB 3.0 Combo PHY DTS Property Configuration Instructions
    - 3.9.4 Android GKI Dependency Property Configuration
- 4. RK3572 USB DRD Mode Switching Commands
- 5. RK3572 USB Firmware Download Function Configuration
  - 5.1 Rockusb3.0 Download Tool Configuration
    - 5.1.1 Windows GUI Tool Configuration
    - 5.1.2 Linux Command Line Tool Configuration
    - 5.1.3 Mac Command Line Tool Configuration
  - 5.2 Rockusb2.0 Download Tool Configuration
  - 5.3 Rockusb2.0 Kernel/U-Boot Configuration
- 6. RK3572 USB 2.0 Low-Power Wakeup System Function Configuration
  - 6.1 RK3572 USB 2.0 Wakeup Function Description
  - 6.2 RK3572 USB 2.0 Wakeup Function Hardware Circuit Design Requirements
  - 6.3 RK3572 USB 2.0 Wakeup Function Software Configuration
- 7. RK3572 Type-C Controller Chip Support List
- 8. Reference Documents

1. Introduction to RK3572 USB Controllers and PHYs

The RK3572 supports 2 independent USB 3.0 DRD controllers, 2 independent USB 2.0 PHYs, and 2 USB3.0/SATA/PCIe Combo PHYs. For more detailed USB controller features, please refer to the RK3572 TRM.

The CCI/MMU/DEBUG functionality design of RK3572 USB is as follows:

1. USB DRD0/1 both support CCI by default to ensure cache coherence, meaning no CPU cache flush operations are required.
2. USB DRD0/1 do not support MMU, meaning USB controller hardware is not allowed to directly access memory space exceeding 4G.
3. USB DRD0 DP/DM supports multiplexing with UART/JTAG. The SDK enables this feature by default and requires use with the RK USB to DEBUG adapter board.

Table 1 Connection Relationship of RK3572 USB Controllers and PHYs

USB Interface Name (Schematic)	USB Controller	USB PHY
USB DRD0	USB0 (DWC3&xHCI)	USB3.0/SATA/PCIe ComboPHY0 + USB2.0 PHY0
USB DRD1	USB1 (DWC3&xHCI)	USB3.0/SATA/PCIe ComboPHY1 + USB2.0 PHY1

Note:

1. USB 3.0 and USB 3.1 Gen1 have the same physical layer transmission rate of 5Gbps. USB 2.0 physical layer transmission rate is 480Mbps;
2. USB3.0/SATA/PCIe Combo PHY only supports 1 set of Tx/Rx. To support Type-C USB 3.0 front/back hardware circuit design, an external USB Switch chip must be added;
3. USB3.0/SATA/PCIe Combo PHY can only support one working mode at a time, meaning USB 3.0 and SATA/PCIe interfaces are mutually exclusive.

The correspondence between RK3572 USB controllers and chip-side USB transmission data pins is shown in Table 2 below.

Table 2 Correspondence between RK3572 USB Interfaces and USB Pins

USB Interface	USB 2.0 Pin	USB 3.0 Pin	VBUSDET	ID
USB DRD0	USB_DRD0_DP/USB_DRD0_DM	USB_DRD0_SSTXP/USB_DRD0_SSTXN USB_DRD0_SSRXP/USB_DRD0_SSRXN	USB_DRD0_VBUSDET	USB_DRD0_ID
USB DRD1	USB_DRD1_DP/USB_DRD1_DM	USB_DRD1_SSTXP/USB_DRD1_SSTXN USB_DRD1_SSRXP/USB_DRD1_SSRXN	USB_DRD1_VBUSDET	USB_DRD1_ID

Note:

1. USB DRD0/1 both support DRD mode, allowing dynamic switching between Device and Host Mode, but Device and Host functions cannot be used simultaneously.
2. USB Host's USB 2.0 Pins and USB 3.0 Pins can be used independently and simultaneously.

3. USB Device's USB 2.0 Pins and USB 3.0 Pins do not support independent simultaneous use.
4. USB3.0 P/N lane polarity inversion is supported.
5. USB_DRD0_VBUSDET is used to detect connection and disconnection in Device mode. For Type-C interfaces with external Type-C controller chips, it is recommended to pull USB_DRD0_VBUSDET up to VCC_3V3_S0. For other circuit design solutions without external Type-C controller chips, USB_DRD0_VBUSDET should still follow the traditional voltage divider circuit design, connected to the VBUS pin of the USB interface, not designed for constant power supply, and cannot be left floating.
6. If USB_DRD1_VBUSDET corresponding to USB DRD1 has a requirement to support Device mode, refer to point 5 for design. If there is only a requirement to support Host mode, USB_DRD1_VBUSDET can be left floating or fixed pulled up to VCC_3V3_S0.
7. USB_DRD0_ID/USB_DRD1_ID are only used for OTG functionality of Micro interface types. If the product does not use the ID pin, leave it floating.
8. The recommended capacitor value for USB 3.0 TXP/TXN series capacitors is 100nF (USB 3.0 Spec requires AC Coupling Capacitor 75 ~ 200nF). If the capacitor value does not meet requirements, it may cause USB 3.0 link training and data communication abnormalities.
9. It is recommended to place 2.2 Ω series resistors on USB 2.0 DP/DM.

The internal connection relationship between RK3572 USB controllers and PHYs, and the corresponding common USB physical interfaces are shown in Figure 1 below.

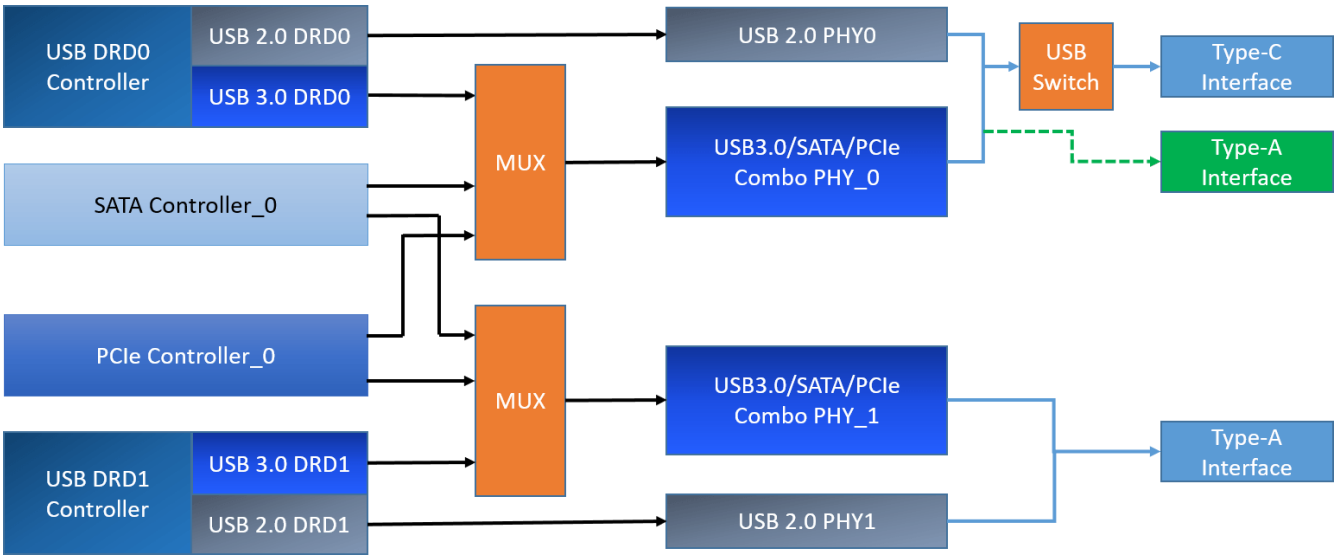


Figure 1 Schematic Diagram of RK3572 USB Controllers and PHYs Connection

Note:

1. The USB interface types supported by RK3572 are not limited to the Type-C/A USB interface types described in Figure 1, but also support all common USB interfaces, including Type-C USB 2.0/3.0, Type-A USB 2.0/3.0, Micro USB 2.0/3.0, etc. To adapt to different USB circuit designs and interface types, the Linux-6.12 kernel USB driver has implemented software compatibility. Developers only need to correctly configure the Linux USB DTS according to the product's USB hardware circuit to enable the corresponding USB interface functions. For detailed USB DTS configuration methods, please refer to RK3572 USB DTS Configuration.
2. USB 3.0 and SATA/PCIe interfaces are mutually exclusive. If Combo PHY0/1 is used for SATA/PCIe interfaces, the corresponding USB controller can only support USB 2.0.

2. RK3572 USB Hardware Circuit Design

2.1 USB Controller Power Supply and Power Management

The power supply for the RK3572 USB controller is VD_LOGIC. Meanwhile, a dedicated power domain for the USB controller is designed internally: RK3572_PD_PHP.

In actual usage scenarios, the Linux USB controller driver will dynamically switch the USB controller's PD on and off based on the USB interface's working conditions using the Linux PM Runtime mechanism to reduce USB controller power consumption. When the system enters secondary standby, to achieve optimal power consumption, the software will forcibly turn off the USB controller's PD. After wakeup from standby, the USB controller driver will reinitialize, and USB peripherals connected to the USB Host interface will also be re-enumerated. In actual product application scenarios, if it is necessary to maintain the USB connection state during secondary standby wakeup, please refer to Redmine USB development materials: https://redmine.rock-chips.com/documents/113

USB Controller Power Control Strategy:

1. For unused USB controllers, it is recommended to configure the corresponding controller DTS node as disabled.
2. For USB controllers enabled in the kernel, when configured as Host mode, the kernel USB driver has supported USB controller Auto suspend functionality (when no peripherals are connected to the USB HOST interface, the controller automatically enters suspend low-power state). Therefore, developers do not need to debug the dynamic power management of the USB controller.
3. For USB controllers enabled in the kernel, when configured as DRD mode, the kernel USB driver supports the PM Runtime mechanism and can dynamically switch the USB controller's PD on and off based on the USB interface connection status.

2.2 USB PHY Power Supply and Power Management

2.2.1 USB 2.0 PHY Power Supply and Power Management

The RK3572 supports 2 independent USB 2.0 PHY0/1. Internally, all USB 2.0 PHYs belong to VD_USBPHY. Meanwhile, the 2 USB 2.0 PHYs share 3 external power supply circuits as shown in Figure 2 below. Therefore, during system operation, it is not possible to reduce USB 2.0 PHY power consumption through simple methods such as hardware power-off and PD shutdown.

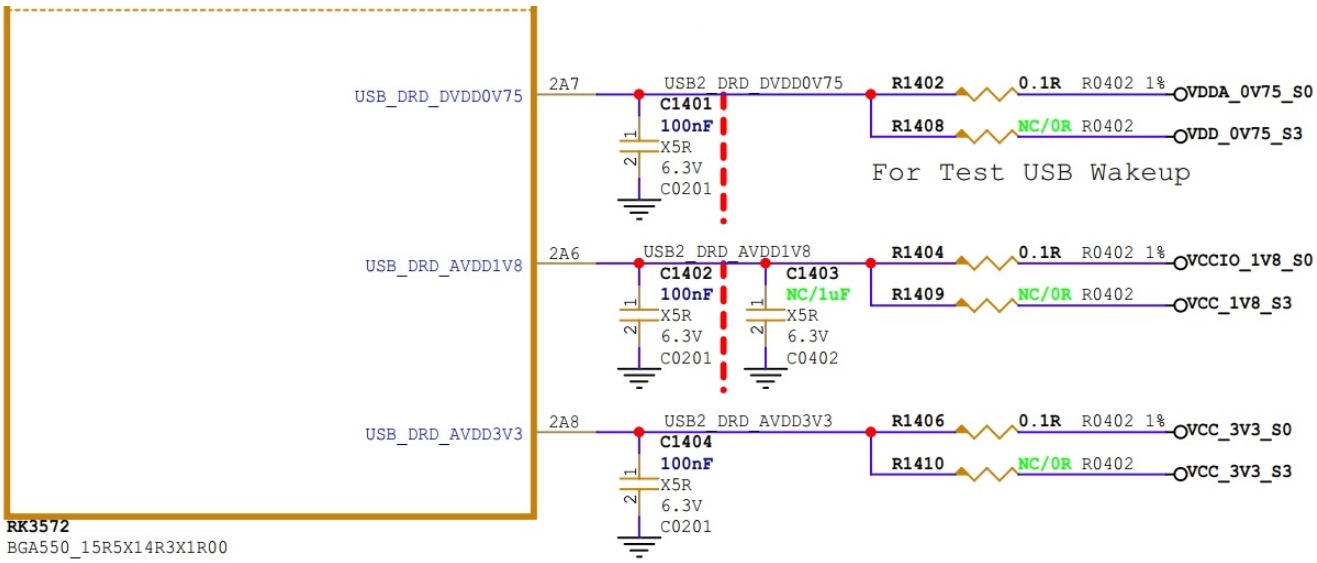


Figure 2 USB 2.0 PHY Power Supply

Table 3 USB 2.0 PHY Supply Voltage Requirements

Power Supply	Min	Nominal	Max	Unit
USB_DRD_DVDD0V75	0.6975	0.75	0.825	V
USB_DRD_AVDD1V8	1.674	1.8	1.98	V
USB_DRD_AVDD3V3	3.069	3.3	3.63	V

Note:

1. In actual circuits, the supply voltage of the USB 2.0 PHY must not exceed ±10% of the nominal value. If it exceeds the specified maximum or falls below the specified minimum, USB communication abnormalities may occur.
2. To support USB wakeup system functionality, the three power supplies related to USB 2.0 PHY need to be merged and connected to the PMU power supply VDD_0V75_S3/VCC_1V8_S3/VCC_3V3_S3 to ensure USB 2.0 PHY remains powered during secondary standby.

USB 2.0 PHY Power Control Strategy:

1. To support Maskrom USB firmware download functionality, it must be ensured that all three power supplies of the USB 2.0 PHY are normal.
2. After system power-on, USB 2.0 PHY0/PHY1 are in Normal mode by default. During the U-Boot SPL stage, the software will actively configure USB 2.0 PHY1 to be in the lowest power IDDQ mode (PHY0 is used for firmware download, no low-power configuration is done in SPL). After entering the system, the kernel USB driver will configure the corresponding USB 2.0 PHY1 to exit IDDQ mode according to application requirements.
3. For USB 2.0 PHY enabled in the kernel, the kernel USB 2.0 PHY driver will automatically perform dynamic power control on the PHY. When USB connection is detected, it will automatically set the USB 2.0 PHY to Normal mode. When USB disconnection is detected, it will automatically set the USB 2.0 PHY to Suspend mode.

Power consumption data of USB 2.0 PHY in different working modes is shown in Table 4 below.

Table 4 USB 2.0 PHY Power Consumption Data (Statistics for a Single USB 2.0 PHY)

Power Supply	Read/Write Data	Dynamic Sleep	PHY Disabled	Secondary Standby	Unit
USB_DRD_DVDD0V75	6.1	2.8	0.16	0	mA
USB_DRD_AVDD1V8	5.1	0.10	0.10	0	mA
USB_DRD_AVDD3V3	2.2	0.12	0.12	0	mA

Note:

1. Read/Write data power consumption test scenario: Connect USB flash drive to copy data, PHY in Normal mode.
2. Dynamic sleep power consumption test scenario: USB 2.0 PHY DTS enabled, but no USB peripheral connected, PHY in Suspend mode.
3. PHY disabled power consumption test scenario: USB 2.0 PHY DTS disabled, PHY in IDDQ mode.
4. Secondary standby power consumption test scenario: All three power supplies of USB 2.0 PHY are turned off.

2.2.2 USB 3.0 PHY Power Supply and Power Management

The RK3572 supports 2 independent USB3.0/SATA/PCIe Combo PHY0/1. Internally, PHY0/1 both belong to PD_BUS (Alive), with no dynamic Powerdown functionality. Externally, PHY0/1 have independent power supplies, as shown in Figure 3.

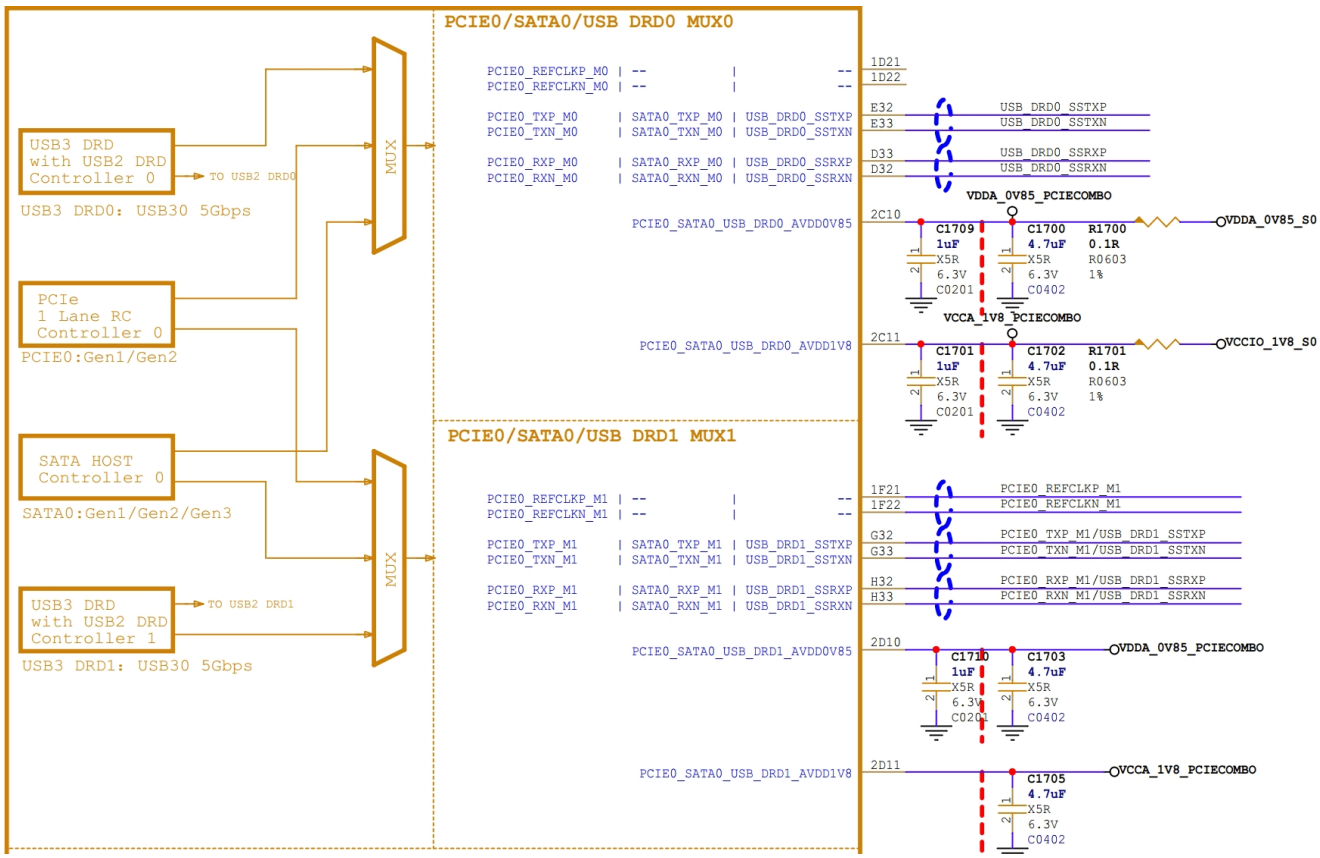


Figure 3 RK3572 USB3.0/SATA/PCIe Combo PHY0/1 Power Supply

Table 5 USB3.0/SATA/PCIe Combo PHY0/1 Supply Voltage Requirements

PHY	Power Supply	Min	Nominal	Max	Unit
Combo PHY0	PCIE0_SATA0_USB_DRD0_AVDD0V85	0.8	0.85	0.935	V
Combo PHY0	PCIE0_SATA0_USB_DRD0_AVDD1V8	1.62	1.8	1.98	V
Combo PHY1	PCIE0_SATA0_USB_DRD1_AVDD0V85	0.8	0.85	0.935	V
Combo PHY1	PCIE0_SATA0_USB_DRD1_AVDD1V8	1.62	1.8	1.98	V

USB3.0/SATA/PCIe Combo PHY Power Control Strategy:

1. When the chip is powered on, USB3.0/SATA/PCIe Combo PHY0/1 are in working state by default. During the U-Boot SPL stage, the software sets Combo PHY0/1 to reset state to keep the PHY at minimum power consumption. After entering the kernel, the USB controller driver will release reset by calling the Combo PHY initialization function.
2. PHY dynamic power control: When Combo PHY works in USB mode, the PHY's PIPE state (P0/P1/P2/P3) is automatically controlled by the USB controller hardware. Depending on different working scenarios, it dynamically enters and exits P0/P1/P2/P3 state. For example, when no USB device is inserted, PIPE is in P3 state; when U3 disk is inserted, it switches to P0 state. When connected to U3 HUB, as long as no other USB peripherals are connected to the downstream port of the HUB, PIPE state will automatically enter P3 low-power state. When a USB peripheral is inserted into the U3 HUB, PIPE state automatically switches to P0. (Note: P0 is normal working state, P3 is lowest power consumption state).
3. When USB3.0/SATA/PCIe Combo interface is explicitly not used, the corresponding Combo PHY can be unpowered, but DTS configuration needs to be modified. For details, please refer to USB3.0 Combo PHY Unpowered DTS Configuration.
4. When Combo PHY is powered, if this PHY is not used, the corresponding PHY DTS node needs to be configured as disabled, which allows the PHY to be in reset state with minimum power consumption.

Table 6 USB 3.0/SATA/PCIe Combo PHY Power Consumption Data

Power Supply	Read/Write Data	Dynamic Sleep	PHY Disabled	Secondary Standby	Unit
AVDD_0V85	44.5	9.6	0.4	0	mA
AVDD_1V8	5.2	0.5	0.2	0	mA

Note:

1. Read/Write data power consumption test scenario: Connect U3 disk to copy data, PHY in P0 state.
2. Dynamic sleep test scenario: PHY DTS enabled, but no USB peripheral connected, PHY in P3 State.
3. PHY disabled test scenario: PHY DTS node configured as disabled, PHY in reset state.
4. Secondary standby power consumption test scenario: Both power supplies of PHY are turned off.

2.3 USB Hardware Circuit Design Considerations

2.3.1 USB DRD0 VBUSDET Circuit Design

USB_DRD0_VBUSDET is used to detect device connection and disconnection status when USB operates as Device. Design considerations are as follows:

1. For Type-C interfaces designed to support PD functionality, i.e., supporting external Type-C controller chips (FUSB302 or HUSB311, etc.), refer to RK3572 EVB1 Type-C circuit design (USB_DRD0_VBUSDET fixed pulled up to VCC_3V3_S0). The software driver can detect USB Device connection and disconnection through the Type-C controller chip's CC.
2. For other circuit design solutions that do not support external Type-C controller chips (such as Type-C USB 2.0 only, Type-A USB 3.0, Micro USB 2.0/3.0), USB_DRD0_VBUSDET is required to follow the traditional voltage divider circuit design, connected to the VBUS pin of the USB interface. VBUSDET is not designed for constant power supply (if USB_DRD0 has USB HOST requirements, independent GPIO or PMIC VBUS control is needed, not multiplexed with USB_DRD1 HOST VBUS control circuit).
3. The high-level range of chip input USB_DRD0_VBUSDET is required to be **[0.9V ~ 3.3V]**.

2.3.2 Maskrom USB Circuit Design

Maskrom fixedly uses USB DRD0 interface for firmware download, supporting up to USB3.0 firmware download, while downward compatible with USB2.0 firmware download. Hardware design requirements are as follows:

1. The three power supplies of USB 2.0 PHY0 (USB_DRD_DVDD0V75/USB_DRD_AVDD1V8/USB_DRD_AVDD3V3) must be powered. For details, please refer to USB 2.0 PHY Power Supply and Power Management.
2. The power supply of USB3.0/SATA/PCIe Combo PHY0 related power supplies can be designed for normal power supply or power-off according to actual project requirements. For details, please refer to USB 3.0 PHY Power Supply and Power Management.
3. To support USB3.0 firmware download, Combo PHY0 related power supplies must be powered, and USB_DRD0_SSTXP/USB_DRD0_SSTXN, USB_DRD0_SSRXP/USB_DRD0_SSRXN must be connected to the USB download port.
4. Maskrom USB enumeration does not depend on the level of USB_DRD0_VBUSDET.
5. For flashing tool configuration and U-Boot configuration instructions, please refer to RK3572 USB Firmware Download Function Configuration.

2.3.3 Type-C USB 3.0 Hardware Circuit

USB3.0/SATA/PCIe Combo PHY only supports 1 set of Tx/Rx. To support Type-C USB 3.0 front/back hardware circuit design, an external USB Switch chip must be added. RK3572 SDK provides two Type-C USB 3.0 hardware circuit reference designs.

1. Type-C USB 3.0 Switch Hardware Circuit Design

   Advantages: High software/hardware maturity, only requires one USB DRD0 interface, simple software control.

   Disadvantages: USB 3.0 Switch chip cost is relatively high, and there are quality requirements for USB 3.0 Switch components. If poor quality components are selected, it may affect USB 3.0 signal quality.

2. Type-C USB 2.0 Switch Hardware Circuit Design

Advantages: USB 2.0 Switch chip cost is relatively low, and USB3.0 signal quality is not affected by the Switch chip.

Disadvantages: Requires both USB DRD0/1 controllers to be occupied simultaneously, with no additional USB controller available for Host use. Meanwhile, software control is more complex, requiring OS and kernel driver coordination.

2.3.3.1 Type-C USB 3.0 Switch Hardware Circuit Design

Taking RK3572 EVB1 Type-C USB 3.0 hardware circuit design as an example, the circuit block diagram is shown below. This chapter mainly explains design considerations. For detailed schematic design, please refer to SDK EVB1 schematic.

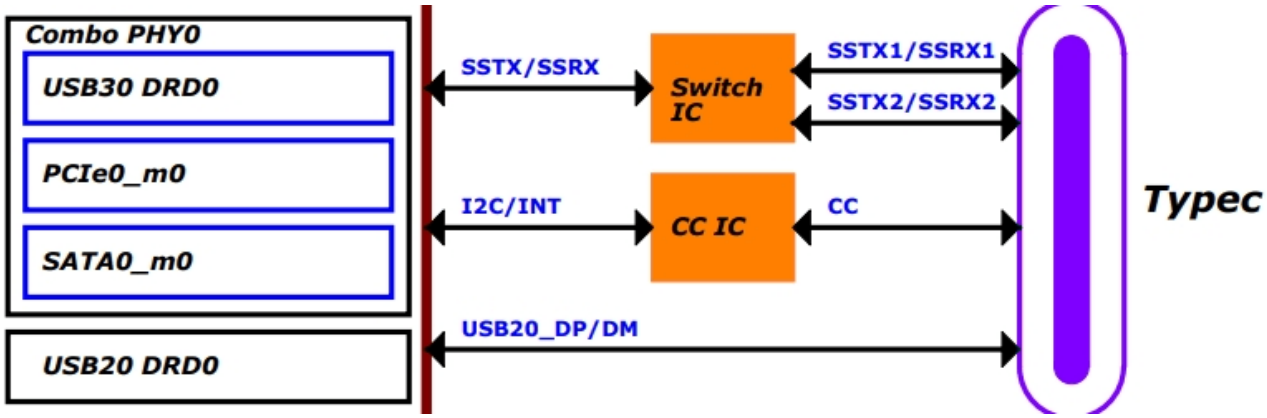


Figure 4 RK3572 Type-C USB 3.0 Switch Hardware Circuit Design Block Diagram

Type-C USB 3.0 Switch hardware circuit design considerations are as follows:

1. The CC IC in Figure 4 is an external Type-C controller chip used to implement Type-C front/back detection and PD charging negotiation. RK3572 supports multiple different models of Type-C controller chips. For details, please refer to Type-C Controller Chip Support List.
2. The Switch IC in Figure 4 is an external USB 3.0 Switch chip used to implement Type-C front/back SSTX1/SSRX1 and SSTX2/SSRX2 switching. Meanwhile, USB 2.0 DP/DM can be directly split into two through PCB traces and connected to the front/back of the Type-C interface.
3. To support high-voltage charging functionality while reducing the overvoltage risk of chip pins, USB_DRD0_VBUSDET should not be directly connected to the VBUS of the Type-C interface. It should be fixed pulled up to 3.3V, but cannot be left floating.
4. USB_DRD0_ID is only used for OTG functionality of Micro interface types. Type-C circuits do not need to use it, leave it floating.
5. For corresponding USB DTS software configuration, please refer to Type-C USB 3.0 Switch Full-Function DTS Configuration.

2.3.3.2 Type-C USB 2.0 Switch Hardware Circuit Design

Taking RK3572 EVB2 Type-C USB 3.0 hardware circuit design as an example, the circuit block diagram is shown below. This chapter mainly explains design considerations. For detailed schematic design, please refer to SDK EVB2 schematic.

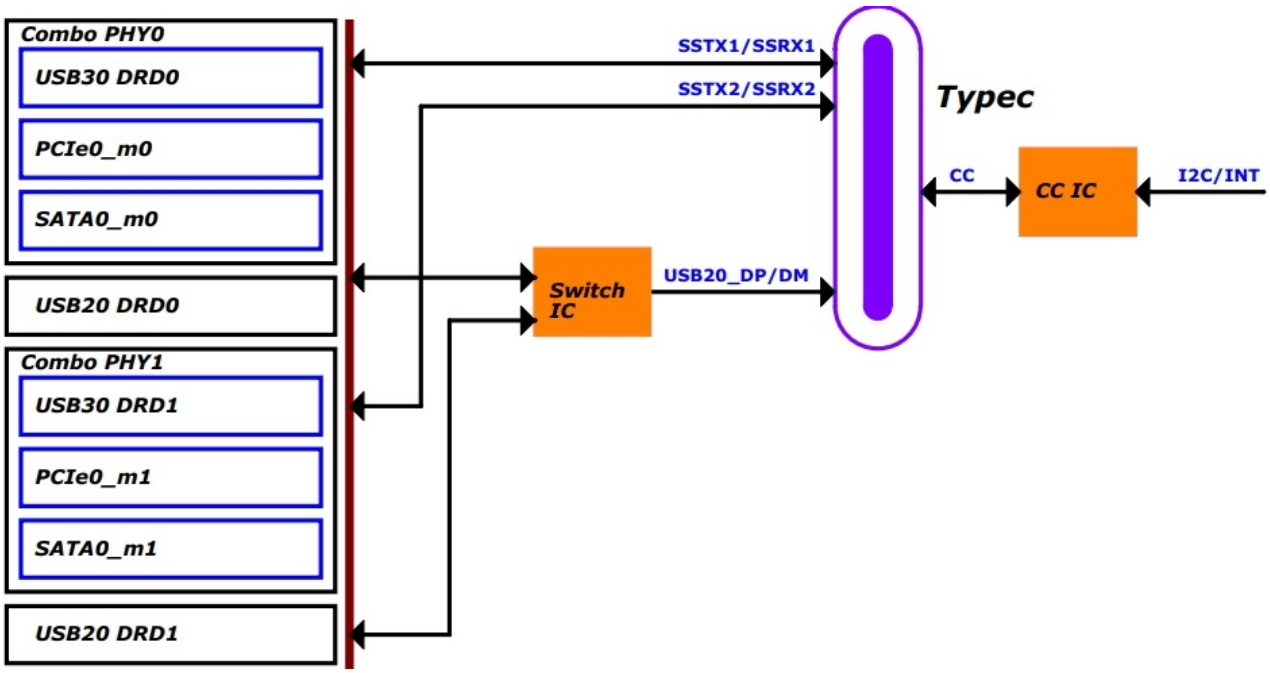
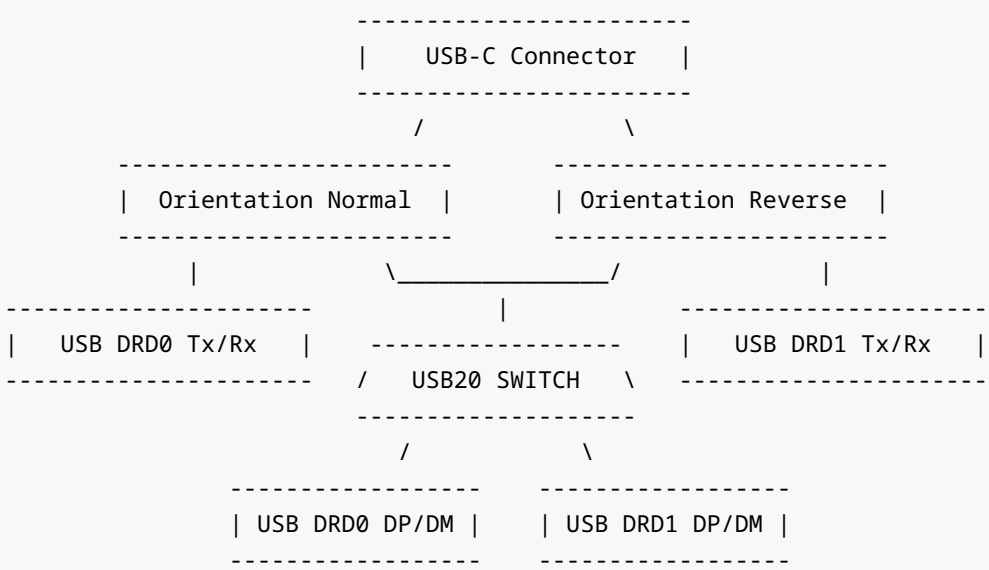


Figure 5 RK3572 Type-C USB 2.0 Switch Hardware Circuit Design Block Diagram

The topology structure of Type-C USB 2.0 Switch is as follows.



Type-C USB 2.0 Switch hardware circuit design considerations are as follows:

1. The SSTX1/SSRX1 of the Type-C interface in Figure 5 are connected to SSTX/SSRX of USB DRD0 controller, while SSTX2/SSRX2 of the Type-C interface are connected to SSTX/SSRX of USB DRD1 controller. It should be noted that hardware design does not support directly splitting the SSTX/SSRX of the same USB DRD controller into two through PCB traces and connecting to the front/back of Type-C, as this will affect USB 5G signal transmission.
2. The CC IC in Figure 5 is an external Type-C controller chip used to implement Type-C front/back detection and PD charging negotiation. RK3572 supports multiple different models of Type-C controller chips. For details, please refer to Type-C Controller Chip Support List.
3. The Switch IC in Figure 5 is an external USB2.0 Switch used to implement Type-C front/back USB DRD0 DP/DM and USB DRD1 DP/DM switching. The purpose of this design is to ensure that USB 3.0 and USB

2.0 interfaces on both front and back of Type-C belong to the same USB DRD controller (e.g., Type-C front SSTX1/SSRX1 and DP/DM both belong to USB DRD0), allowing the USB controller hardware to automatically execute USB 3.0 and USB 2.0 mode switching.

4. To support high-voltage charging functionality while reducing the overvoltage risk of chip pins, USB_DRD0_VBUSDET should not be directly connected to the VBUS of the Type-C interface. It should be fixed pulled up to 3.3V, but cannot be left floating.
5. USB_DRD0_ID is only used for OTG functionality of Micro interface types. Type-C circuits do not need to use it, leave it floating.
6. For corresponding USB DTS software configuration, please refer to Type-C USB 2.0 Switch Full-Function DTS Configuration.

2.3.4 Type-C USB 2.0 with PD Hardware Circuit

Taking RK3572 Tablet DEMO1 Type-C USB 2.0 with PD hardware circuit design as an example, the circuit block diagram is shown below. This chapter mainly explains design considerations. For detailed schematic design, please refer to SDK DEMO1 schematic.

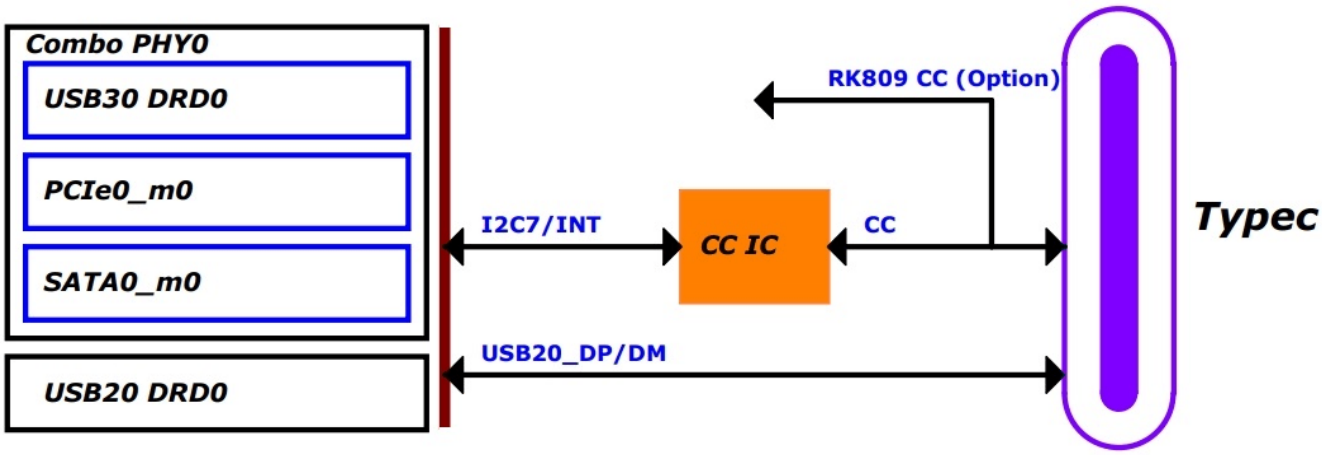


Figure 6 RK3572 Type-C USB 2.0 with PD Hardware Circuit Design Block Diagram

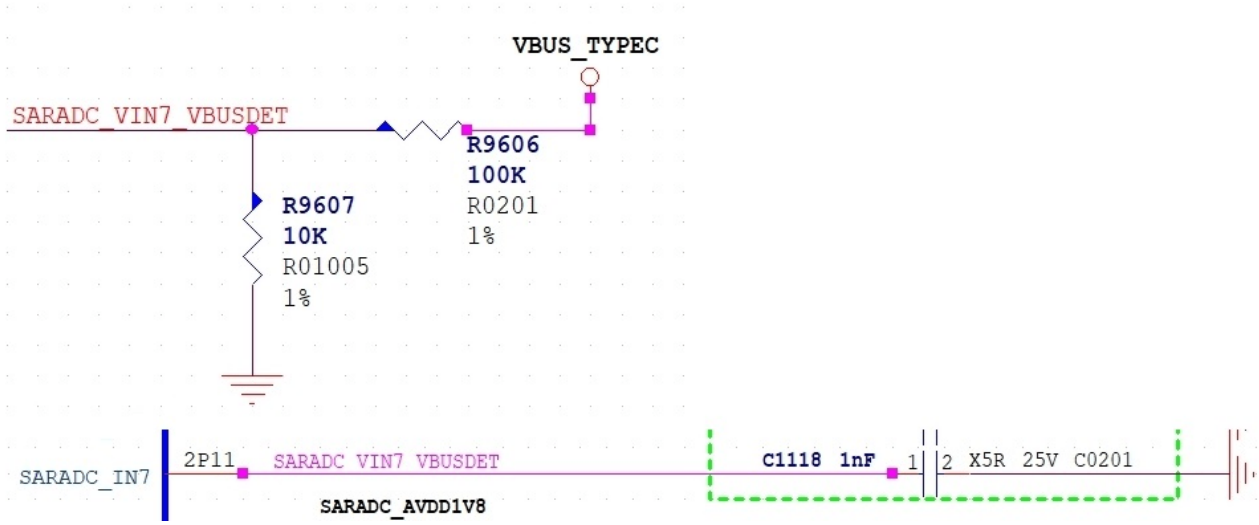


Figure 7 RK3572 SARADC VBUS Detection Circuit

Type-C USB 2.0 with PD hardware circuit design considerations are as follows:

1. The CC IC in Figure 6 is an external Type-C controller chip used to implement PD charging negotiation. Tablet DEMO1 provides two CC hardware design solutions. In addition to the conventional CC IC design solution (HUSB311), it also supports PMIC RK809C PD negotiation. If RK809C is selected for PD negotiation, one channel of SARADC is required to sample the VBUS voltage of the Type-C interface through a voltage divider circuit, which is used for VSAFE0V detection in Type-C certification test TD.4.7.2. Refer to the circuit shown in Figure 7.
2. USB 2.0 DP/DM can be directly split into two through PCB traces and connected to the front/back of the Type-C interface, without requiring USB2.0 Switch.
3. Combo PHY0 can be used for PCIe0 or SATA0. If neither PCIe0 nor SATA0 is used, Combo PHY0 is allowed to be unpowered, and the corresponding power supply pins PCIE0_SATA0_USB_DRD0_AVDD0V85/PCIE0_SATA0_USB_DRD0_AVDD1V8 are grounded.
4. To support high-voltage charging functionality while reducing the overvoltage risk of chip pins, USB_DRD0_VBUSDET should not be directly connected to the VBUS of the Type-C interface. It should be fixed pulled up to 3.3V, but cannot be left floating.
5. USB_DRD0_ID is only used for OTG functionality of Micro interface types. Type-C circuits do not need to use it, leave it floating.
6. For corresponding USB DTS software configuration, please refer to Type-C USB 2.0 with PD DTS Configuration.

2.3.5 Type-C USB 2.0 only Hardware Circuit

Taking RK3572 IOTEST Type-C USB 2.0 only hardware circuit design as an example, the circuit block diagram is shown below. This chapter mainly explains design considerations. For detailed schematic design, please refer to SDK IOTEST schematic.

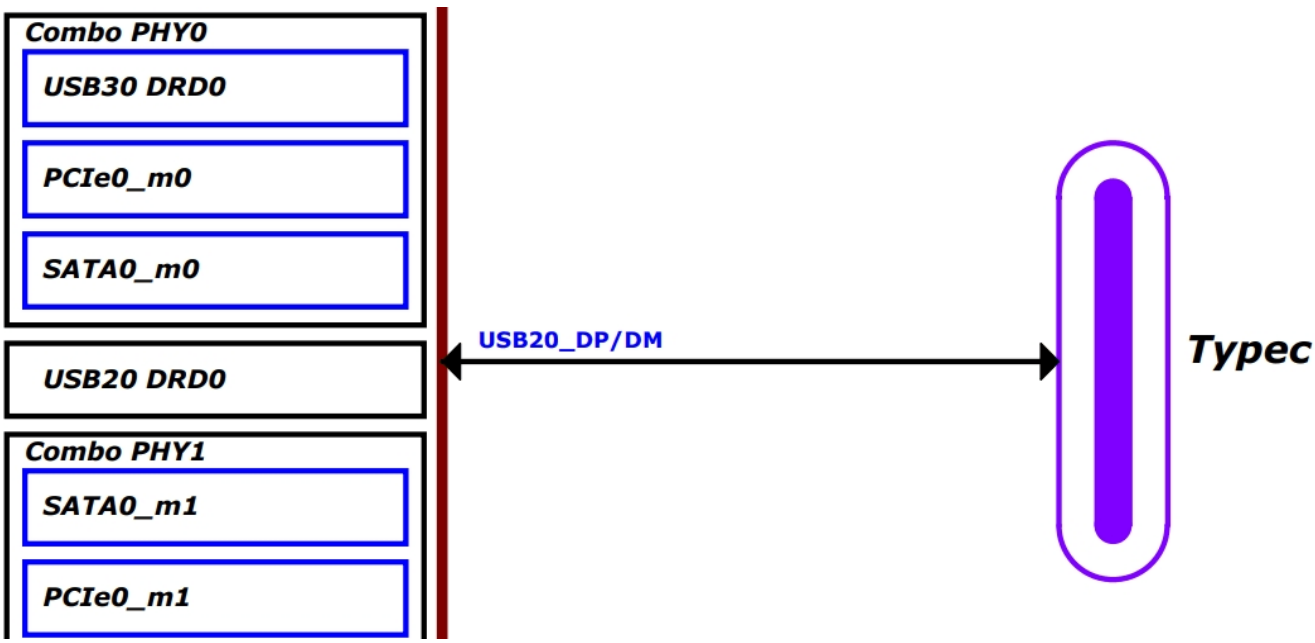


Figure 8 RK3572 Type-C USB 2.0 without PD Hardware Circuit Design Block Diagram

Type-C USB 2.0 without PD hardware circuit design considerations are as follows:

1. This hardware solution does not support PD high-voltage high-current negotiation. USB_DRD0_VBUSDET is required to follow the traditional voltage divider circuit design, connected to the VBUS pin of the Type-C interface, not designed for constant power supply, and cannot be left floating.

2. This hardware solution does not support automatic USB DFP/UFP switching. If DFP/UFP switching is required, it is recommended to use RK3572 USB DRD Mode Switching Commands.
3. USB 2.0 DP/DM can be directly split into two through PCB traces and connected to the front/back of the Type-C interface, without requiring USB2.0 Switch.
4. Combo PHY0 can be used for PCIe0 or SATA0. If neither PCIe0 nor SATA0 is used, Combo PHY0 is allowed to be unpowered, and the corresponding power supply pins PCIE0_SATA0_USB_DRD0_AVDD0V85/PCIE0_SATA0_USB_DRD0_AVDD1V8 are grounded.
5. USB_DRD0_ID is only used for OTG functionality of Micro interface types. Type-C circuits do not need to use it, leave it floating.
6. If compatibility with USB-C to C cables is required, and the product only has USB Device mode application requirements, it is recommended to pull down the CC1/CC2 pins of the Type-C interface to ground with 5.1KΩ resistors.
7. For corresponding USB DTS software configuration, please refer to Type-C USB 2.0 only DTS Configuration.

2.3.6 Type-A USB 3.0 Hardware Circuit

Taking RK3572 EVB1 Type-A USB 3.0 hardware circuit design as an example, the circuit block diagram is shown below. This chapter mainly explains design considerations. For detailed schematic design, please refer to SDK EVB1 schematic.

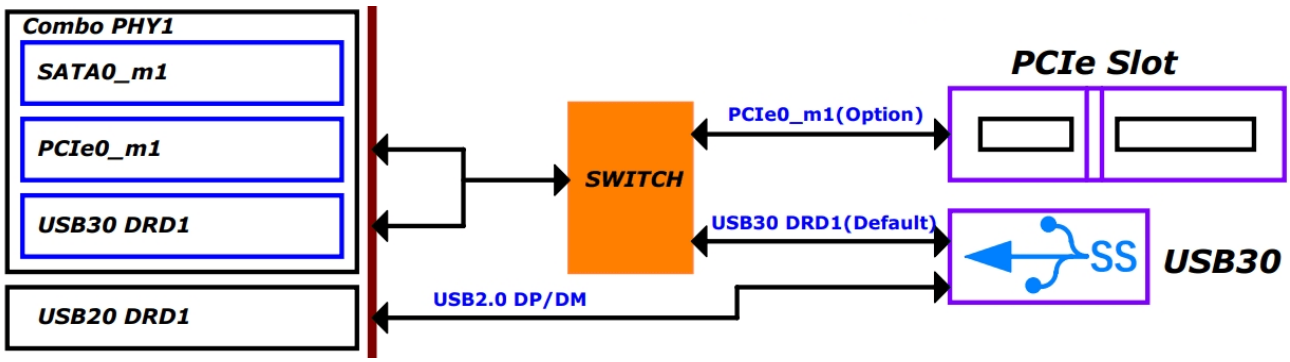


Figure 9 RK3572 Type-A USB 3.0 Hardware Circuit Design Block Diagram

Type-A USB 3.0 hardware circuit design considerations are as follows:

1. The SWITCH IC in Figure 9 is used to switch between USB 3.0 path and PCIe path of Combo PHY. However, the software driver provided by SDK does not support dynamic switching between PCIe and USB 3.0 functions. If switched to PCIe path on hardware, DTS needs to be reconfigured and kernel recompiled. For DTS modification, please refer to SDK code project `arch/arm64/boot/dts/rockchip/rk3572-evb1-v10-pcie.dts`.
2. USB_DRD1_VBUSDET is used for connection and disconnection detection in Device mode. EVB1 Type-A USB 3.0 is fixedly configured as Host mode, USB_DRD1_VBUSDET is not needed, and can be externally pulled up to 3.3V or left floating.
3. USB_DRD1_ID is only used for OTG functionality of Micro interface types. Type-A circuits do not need to use it, leave it floating.
4. For corresponding USB DTS software configuration, please refer to Type-A USB 3.0 DTS Configuration.

2.3.7 Multi Type-A USB 3.0 Hardware Circuit

Taking RK3572 EVB3 supporting 2 x Type-A USB3.0 + 3 x Type-A USB2.0 hardware circuit design as an example, the circuit block diagram is shown below. This chapter mainly explains design considerations. For detailed schematic design, please refer to SDK EVB3 schematic.

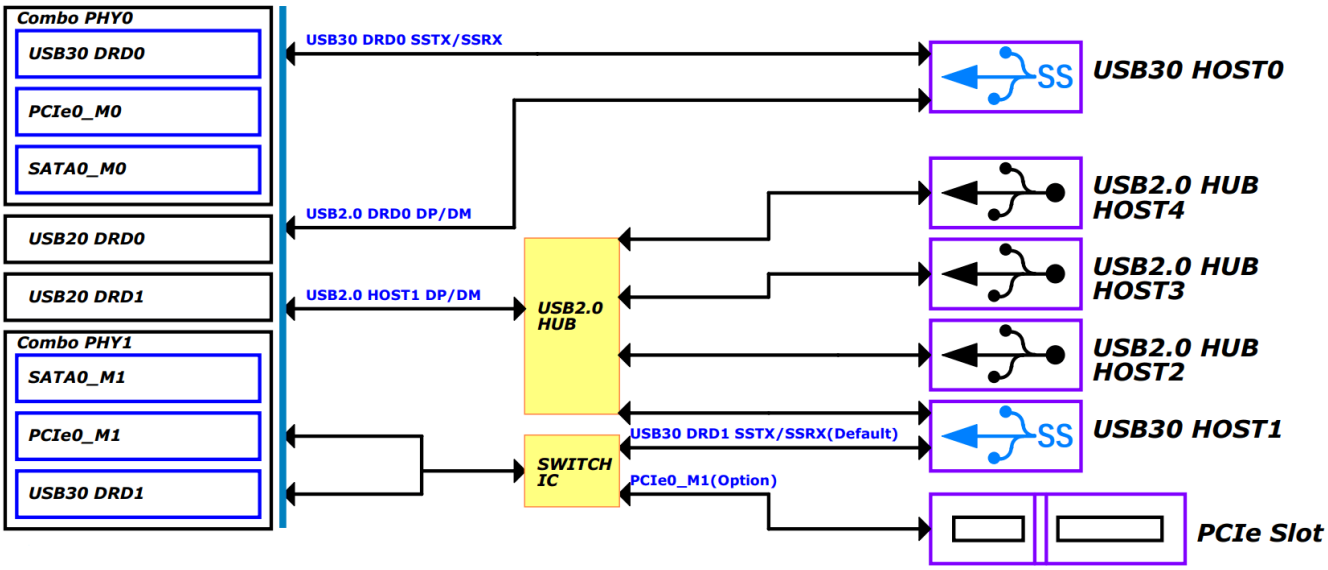


Figure 10 RK3572 Multi Type-A USB 3.0 Hardware Circuit Design Block Diagram

Multi Type-A USB 3.0 hardware circuit design considerations are as follows:

1. RK3572 supports two complete USB 3.0 DRD controllers. When operating as USB Host, USB 2.0 Pins and USB 3.0 Pins support split independent simultaneous use. When operating as USB Device, USB 2.0 Pins and USB 3.0 Pins do not support independent simultaneous use. Considering that USB DRD0 usually needs to support Device mode (such as: firmware download/ADB debugging), in hardware design, USB 2.0 Pins and USB 3.0 Pins of USB DRD0 are required to be connected to the same Type-A USB 3.0 interface and cannot be split for use.
2. When USB DRD1 is fixedly used as Host mode, the corresponding USB 2.0 Pins and USB 3.0 Pins can be split for use. Among them, USB 2.0 can expand multiple USB 2.0 interfaces through USB 2.0 HUB, and one downstream USB 2.0 port of the HUB can be combined with USB 3.0 Pins of USB DRD1 to form a complete Type-A USB 3.0 interface.
3. USB_DRD0_VBUSDET is used for connection and disconnection detection in Device mode. It is required to follow the traditional voltage divider circuit design and connected to the corresponding Type-A interface VBUS pin. USB_DRD1_VBUSDET does not need to support connection and disconnection detection in Device mode, and can be externally pulled up to 3.3V or left floating.
4. The SWITCH IC in Figure 10 is used to switch between USB 3.0 path and PCIe path of Combo PHY. However, the software driver provided by SDK does not support dynamic switching between PCIe and USB 3.0 functions. If switched to PCIe path on hardware, DTS needs to be reconfigured and kernel recompiled. For DTS modification, please refer to SDK code project `arch/arm64/boot/dts/rockchip/rk3572-evb1-v10-pcie.dts`.
5. USB_DRD0_ID/USB_DRD1_ID are only used for OTG functionality of Micro interface types. Type-A circuits do not need to use them, leave them floating.
6. For corresponding USB DTS software configuration, please refer to Multi Type-A USB 3.0 DTS Configuration.

3. RK3572 USB DTS Configuration

RK3572 USB DTS configuration includes: chip-level USB controller/PHY DTSI configuration and board-level DTS configuration. For detailed configuration methods, please refer to the following kernel documents:

1. kernel/Documentation/devicetree/bindings/usb/snps,dwc3.yaml (USB Controller)
2. kernel/Documentation/devicetree/bindings/phy/phy-rockchip-inno-usb2.yaml (USB 2.0 PHY)
3. kernel/Documentation/devicetree/bindings/phy/phy-rockchip-naneng-combphy.yaml (USB 3.0 Combo PHY)
4. kernel/Documentation/devicetree/bindings/connector/usb-connector.yaml (Type-C connector)
5. kernel/Documentation/devicetree/bindings/usb/hynetek,husb311.yaml (HUSB311 TCPC Chip)
6. kernel/Documentation/devicetree/bindings/usb/rockchip,pmic-tcpc.yaml (RK809C TCPC Chip)

3.1 USB Chip-Level DTSI Configuration

The main nodes related to USB controllers and PHYs in the RK3572 DTSI file are shown in the table below. Since the USB DTSI nodes configure public resources and attributes of USB controllers and PHYs, it is recommended that developers do not modify them. The complete path of the corresponding DTSI is as follows:

`arch/arm64/boot/dts/rockchip/rk3572.dtsi`

Table 7 Correspondence between RK3572 USB Interfaces and USB DTS Nodes

USB Interface Name (Schematic)	USB Controller DTS Node	USB PHY DTS Node
USB DRD0	usb_drd0_dwc3	u2phy0 u2phy0_otg combphy0_psu
USB DRD1	usb_drd1_dwc3	u2phy1 u2phy1_otg combphy1_psu

3.1.1 USB Controller DTSI Configuration

```
/* USB DRD0 Controller (USB2.0/3.0) */
usb_drd0_dwc3: usb@24000000 {
        compatible = "rockchip,rk3572-dwc3", "rockchip,rk3576-dwc3", "snps,dwc3";
        ......
};

/* USB DRD1 Controller (USB2.0/3.0) */
usb_drd1_dwc3: usb@24400000 {
        compatible = "rockchip,rk3572-dwc3", "rockchip,rk3576-dwc3", "snps,dwc3";
        ......
};
```

The USB controller supports LPM functionality and dynamic suspend USB2/USB3 PHY functionality by default, which can reduce the operating power consumption of the USB module. Meanwhile, it supports configuring to disable the corresponding functions through DTS properties. For detailed DTS configuration instructions, please refer to USB Controller DTS Property Configuration Instructions.

3.1.2 USB PHY DTSI Configuration

USB 2.0/3.0 PHYs and USB controllers have a one-to-one mapping relationship, need to be configured in pairs, and do not support swapping the mapping between PHY and controller. For the internal connection relationship between USB PHYs and controllers, please refer to Introduction to RK3572 USB Controllers and PHYs. In the DTSI node, the USB controller associates the corresponding USB PHY through the node's "phys" property.

```
/* USB 2.0 PHY0/1 */
usb2phy_grf: syscon@26054000 {
        compatible = "rockchip,rk3572-usb2phy-grf", "syscon", "simple-mfd";
        reg = <0x0 0x26054000 0x0 0x4000>;
        ......
        /* PHY0 corresponds to USB DRD0 controller */
        u2phy0: usb2-phy@0 {
                compatible = "rockchip,rk3572-usb2phy";
                reg = <0x0 0x10>;
                ......
                u2phy0_otg: otg-port {
                        ......
                };
        };
        /* PHY1 corresponds to USB DRD1 controller */
        u2phy1: usb2-phy@2000 {
                compatible = "rockchip,rk3572-usb2phy";
                reg = <0x2000 0x10>;
                ......
                u2phy1_otg: otg-port {
                        ......
                };
        };
};

/* USB3.0/SATA/PCIe Combo PHY0, corresponds to USB DRD0 controller */
combphy0_psu: phy@2c410000 {
        compatible = "rockchip,rk3572-pipe-combphy";
        reg = <0x0 0x2c410000 0x0 0x100>;
        ......
};

/* USB3.0/SATA/PCIe Combo PHY1, corresponds to USB DRD1 controller */
combphy1_psu: phy@2c420000 {
        compatible = "rockchip,rk3572-pipe-combphy";
        reg = <0x0 0x2c420000 0x0 0x100>;
        ......
};
```

When USB 2.0 PHY operates in Device mode, it supports charging detection and dynamically entering suspend functionality by default. Meanwhile, it supports configuring to disable the corresponding functions through DTS properties. For detailed USB 2.0 PHY property configuration instructions, please refer to USB 2.0 PHY DTS Property Configuration Instructions.

For USB 3.0/SATA/PCIe Combo PHY0/1 property configuration instructions, please refer to USB 3.0 Combo PHY DTS Property Configuration Instructions.

3.2 Type-C USB 3.0 Switch Full-Function DTS Configuration

Refer to the DTS configuration of the Type-C USB 3.0 interface in `arch/arm64/boot/dts/rockchip/rk3572-evb1-v10.dtsi`.

```
/* Configure USB3.0 Switch, "select-gpios" corresponds to the hardware SEL Pin, used to
support Type-C front/back Tx/Rx switching */
usb3_switch: usb3-switch {
        compatible = "broadchip,bct4360", "gpio-sbu-mux";
        select-gpios = <&gpio1 RK_PD4 GPIO_ACTIVE_HIGH>;
        pinctrl-names = "default";
        pinctrl-0 = <&usb3_switch_default>;
        orientation-switch;
};

/* Enable USB3.0/SATA/PCIe Combo PHY0 */
&combphy0_psu {
        status = "okay";
};

/* USB2.0 PHY parent node configures property "orientation-switch", used to register Type-C
orientation switch callback function and control Bvalid signal */
&u2phy0 {
        orientation-switch;
        status = "okay";
};

/* USB2.0 PHY child node configures property "rockchip,typec-vbus-det", indicating support
for hardware design where Type-C VBUSDET pin is always pulled high */
&u2phy0_otg {
        rockchip,typec-vbus-det;
        status = "okay";
};

/* Configure USB3.0 DRD0 Controller */
&usb_drd0_dwc3 {
        dr_mode = "otg";
        usb-role-switch; /* Enable registration of Type-C USB Role Switch callback function
for switching DFP/UFP */
        status = "okay";
};
```

```
/* Configure external Type-C controller chip HUSB311 */
&i2c1 { /* The actual hardware circuit design does not restrict HUSB311 to be connected to
I2C1. It needs to be configured according to the actual I2C bus it is connected to */
        status = "okay";

        usbc0: husb311@4e {
                compatible = "hynetek,husb311";
                reg = <0x4e>; /* Configure I2C address of HUSB311 */
                interrupt-parent = <&gpio0>; /* Configure the GPIO corresponding to the
interrupt pin of HUSB311 */
                interrupts = <RK_PA5 IRQ_TYPE_LEVEL_LOW>; /* Configure low level to
generate interrupt */
                pinctrl-names = "default";
                pinctrl-0 = <&usbc0_int>;
                vbus-supply = <&vbus5v0_typec>; /* Configure Type-C VBUS property, used to
control output 5V to peripherals */
                status = "okay";

                /* Detailed connector configuration, refer to kernel documentation usb-
connector.yaml */
                usb_con0: connector {
                        compatible = "usb-c-connector";
                        label = "USB-C";
                        data-role = "dual"; /* Support both host and device data roles */
                        power-role = "dual"; /* Support both source and sink power roles */
                        try-power-role = "sink"; /* Prefer sink role */
                        op-sink-microwatt = <1000000>; /* Operating power required by the
sink, in microwatts */
                        orientation-switch = <&usb3_switch &u2phy0>; /* Configure orien
switch linked nodes */
                        usb-role-switch = <&usb_drd0_dwc3>; /* Configure role switch linked
node */

                        /* Configure actual supported sink and source capabilities */
                        sink-pdos = <PDO_FIXED(5000, 1000, PDO_FIXED_DATA_SWAP |
PDO_FIXED_USB_COMM | PDO_FIXED_EXTPOWER | PDO_FIXED_DUAL_ROLE)>;
                        source-pdos = <PDO_FIXED(5000, 3000, PDO_FIXED_DATA_SWAP |
PDO_FIXED_USB_COMM | PDO_FIXED_EXTPOWER | PDO_FIXED_DUAL_ROLE)>;
                };
        };
};
```

Note: Software modifications for replacing HUSB311 chip with FUSB302/AW35615

1. DTS modification: Only simple modifications based on the DTS configuration of HUSB311 are needed. Reference modifications are as follows:

```
/* Configure external Type-C controller chip FUSB302 */
&i2c2 {
        usbc0: fusb302@22 {
                compatible = "fcs,fusb302";
                reg = <0x22>; /* Configure I2C address of FUSB302 */
                ...... /* Other configurations are the same as HUSB311 */
        };
};
```

2. Driver software modification: Modify the irq wakeup mechanism of the FUSB302 driver to support various application scenarios such as VBUS power off or power on during standby.

```
diff --git a/drivers/usb/typec/tcpm/fusb302.c b/drivers/usb/typec/tcpm/fusb302.c
index 77a050cd2e88..72bef94887b4 100644
--- a/drivers/usb/typec/tcpm/fusb302.c
+++ b/drivers/usb/typec/tcpm/fusb302.c
@@ -105,6 +105,7 @@ struct fusb302_chip {
        bool vbus_on;
        bool charge_on;
        bool vbus_present;
+       bool wakeup;
        enum typec_cc_polarity cc_polarity;
        enum typec_cc_status cc1;
        enum typec_cc_status cc2;
@@ -1769,7 +1770,8 @@ static int fusb302_probe(struct i2c_client *client,
                dev_err(dev, "cannot request IRQ for GPIO Int_N, ret=%d", ret);
                goto tcpm_unregister_port;
        }
-       enable_irq_wake(chip->gpio_int_n_irq);
+       chip->wakeup = device_property_read_bool(dev, "wakeup-source");
+       device_init_wakeup(dev, true);
        i2c_set_clientdata(client, chip);

        return ret;
@@ -1788,7 +1790,7 @@ static void fusb302_remove(struct i2c_client *client)
        struct fusb302_chip *chip = i2c_get_clientdata(client);

-       disable_irq_wake(chip->gpio_int_n_irq);
+       device_init_wakeup(chip->dev, false);
        free_irq(chip->gpio_int_n_irq, chip);
        kthread_destroy_worker(chip->irq_worker);
        cancel_delayed_work_sync(&chip->bc_lvl_handler);
@@ -1809,6 +1811,11 @@ static int fusb302_pm_suspend(struct device *dev)

        /* Make sure any pending irq_work is finished before the bus suspends */
        kthread_flush_worker(chip->irq_worker);
+
+       if (device_may_wakeup(dev) && (!chip->vbus_on || chip->wakeup))
+               enable_irq_wake(chip->gpio_int_n_irq);
+       else
+               disable_irq(chip->gpio_int_n_irq);
```

```
        return 0;
 }

@@ -1819,6 +1826,11 @@ static int fusb302_pm_resume(struct device *dev)
        u8 pwr;
        int ret = 0;

+       if (device_may_wakeup(dev) && (!chip->vbus_on || chip->wakeup))
+               disable_irq_wake(chip->gpio_int_n_irq);
+       else
+               enable_irq(chip->gpio_int_n_irq);
+
        /*
         * When the power of fusb302 is lost or i2c read failed in PM S/R
         * process, we must reset the tcpm port first to ensure the devices
```

3.3 Type-C USB 2.0 Switch Full-Function DTS Configuration

Refer to the DTS configuration of the Type-C USB 3.0 interface in `arch/arm64/boot/dts/rockchip/rk3572-evb2-v10.dtsi`.

```
/* Configure extcon usb driver, used to receive TCPM orien/role notifications and send usb
role extcon to the corresponding USB DRD0/1 controller drivers */
extcon_usb: extcon-usb {
        compatible = "rockchip,extcon-usb";
        orientation-switch;
        usb-role-switch;

        extcon_usb_port0: extcon-usb-port0 {
                rockchip,usbc-orientation = <1>; /* Configure extcon port0 corresponding to
normal orien */
                extcon = <&u2phy0>; /* Fixed reference to the extcon dev created by u2phy0,
used to notify u2phy of the current usb role */
        };

        extcon_usb_port1: extcon-usb-port1 {
                rockchip,usbc-orientation = <2>; /* Configure extcon port1 corresponding to
reverse orien */
                extcon = <&u2phy0>; /* Fixed reference to the extcon dev created by u2phy0,
used to notify u2phy of the current usb role. Note that u2phy1 cannot be referenced */
        };
};

/* Configure USB2.0 Switch, "select-gpios" corresponds to the hardware SEL Pin, used to
support Type-C front/back DP/DM switching */
usb2_switch: usb2-switch {
        compatible = "broadchip,bct4010", "gpio-sbu-mux";
        select-gpios = <&gpio1 RK_PD4 GPIO_ACTIVE_HIGH>; /* GPIO can select other pins, but
requires default pull-down at power-on, i.e. default select DRD0 */
        pinctrl-names = "default";
        pinctrl-0 = <&usb2_switch_default>;
```

```
        orientation-switch;
};

/* Enable USB3.0/SATA/PCIe Combo PHY0, used for Type-C front TX1/RX1 */
&combphy0_psu {
        status = "okay";
};
/* Enable USB3.0/SATA/PCIe Combo PHY1, used for Type-C back TX2/RX2 */
&combphy1_psu {
        status = "okay";
};

/* USB2.0 PHY0 parent node configures property "orientation-switch", used to register Type-C orientation switch callback function and control Bvalid signal */
&u2phy0 {
        orientation-switch;
        status = "okay";
};

/* USB2.0 PHY1 parent node configures property "rockchip,partner-phy", indicating combination with PHY0 as Type-C partner port */
&u2phy1 {
        rockchip,partner-phy = <&u2phy0>;
        extcon = <&u2phy0>; /* Fixed reference to the extcon dev created by u2phy0, to unify the USB BC1.2 extcon notification chain */
        status = "okay";
};

/* USB2.0 PHY0 child node configures property "rockchip,typec-vbus-det", indicating support for hardware design where Type-C VBUSDET pin is always pulled high */
&u2phy0_otg {
        rockchip,typec-vbus-det;
        status = "okay";
};

/* USB2.0 PHY1 child node configures property "rockchip,typec-vbus-det", indicating support for hardware design where Type-C VBUSDET pin is always pulled high */
&u2phy1_otg {
        rockchip,typec-vbus-det;
        status = "okay";
};

/* USB3.0 DRD0 Controller, property "extcon" is used to receive notifications from extcon usb driver port0 to switch DFP/UFP */
&usb_drd0_dwc3 {
        dr_mode = "otg";
        extcon = <&extcon_usb_port0>;
        status = "okay";
};

/* USB3.0 DRD1 Controller, property "extcon" is used to receive notifications from extcon usb driver port1 to switch DFP/UFP */
&usb_drd1_dwc3 {
```

```
        dr_mode = "otg";
        extcon = <&extcon_usb_port1>;
        status = "okay";
};

/* Configure external Type-C controller chip HUSB311 */
&i2c1 { /* The actual hardware circuit design does not restrict HUSB311 to be connected to
I2C1. It needs to be configured according to the actual I2C bus it is connected to */
        status = "okay";

        usbc0: husb311@4e {
                compatible = "hynetek,husb311";
                reg = <0x4e>; /* Configure I2C address of HUSB311 */
                interrupt-parent = <&gpio0>; /* Configure the GPIO corresponding to the
interrupt pin of HUSB311 */
                interrupts = <RK_PC1 IRQ_TYPE_LEVEL_LOW>; /* Configure low level to
generate interrupt */
                pinctrl-names = "default";
                pinctrl-0 = <&usbc0_int>;
                vbus-supply = <&vbus5v0_typec>; /* Configure Type-C VBUS property, used to
control output 5V to peripherals */
                status = "okay";

                /* Detailed connector configuration, refer to kernel documentation usb-
connector.yaml */
                usb_con0: connector {
                        compatible = "usb-c-connector";
                        label = "USB-C";
                        data-role = "dual"; /* Support both host and device data roles */
                        power-role = "dual"; /* Support both source and sink power roles */
                        try-power-role = "sink";  /* Prefer sink role */
                        op-sink-microwatt = <1000000>; /* Operating power required by the
sink, in microwatts */
                        orientation-switch = <&extcon_usb &usb2_switch &u2phy0>; /*
Configure orien switch linked nodes, supports up to 3 */
                        usb-role-switch = <&extcon_usb>; /* Configure role switch linked
node */

                        /* Configure actual supported sink and source capabilities */
                        sink-pdos = <PDO_FIXED(5000, 1000, PDO_FIXED_DATA_SWAP |
PDO_FIXED_USB_COMM | PDO_FIXED_EXTPOWER | PDO_FIXED_DUAL_ROLE)>;
                        source-pdos = <PDO_FIXED(5000, 3000, PDO_FIXED_DATA_SWAP |
PDO_FIXED_USB_COMM | PDO_FIXED_EXTPOWER | PDO_FIXED_DUAL_ROLE)>;
                };
        };
};
```

3.4 Type-C USB 2.0 with PD DTS Configuration

Refer to the DTS configuration of the Type-C USB 2.0 interface in `arch/arm64/boot/dts/rockchip/rk3572-tablet-v10.dts`.

```
/* Configure USB3.0/SATA/PCIe Combo PHY0 */
&combphy0_psu {
        rockchip,dis-u3otg0-port; /* If SATA/PCIe uses Combo PHY0, this property needs to
be added */
        status = "okay"; /* If SATA/PCIe does not use Combo PHY0, change this to status =
"disabled" */
};

/* Configure external Type-C controller chip RK809C */
&i2c0 { /* The actual hardware circuit design does not restrict RK809C to be connected to
I2C0. It needs to be configured according to the actual I2C bus it is connected to */
        status = "okay";

        usbc0: rk809c-tcpc@22 {
                compatible = "rockchip,rk809c-tcpc", "rockchip,rk817b2-tcpc";
                reg = <0x22>; /* Configure I2C address of RK809C */
                interrupt-parent = <&gpio0>; /* Configure the GPIO corresponding to the
interrupt pin of RK809C */
                interrupts = <RK_PA6 IRQ_TYPE_LEVEL_LOW>; /* Configure low level to
generate interrupt */
                pinctrl-names = "default";
                pinctrl-0 = <&pmic_pins>;
                vbus-supply = <&vbus5v0_typec>; /* Configure Type-C VBUS property, used to
control output 5V to peripherals */
                io-channels = <&saradc 7>; /* Configure SARADC corresponding to Type-C VBUS
*/
                io-channel-names = "vsafe0v-detect";
                /* rockchip,vsafe0v-det; Enable vsafe0v detection, optional property, used
for vsafe0v related item testing in PD certification test */
                status = "okay";

                /* Detailed connector configuration, refer to kernel documentation usb-
connector.yaml */
                usb_con: connector {
                        compatible = "usb-c-connector";
                        label = "USB-C";
                        data-role = "dual"; /* Support both host and device data roles */
                        power-role = "dual"; /* Support both source and sink power roles */
                        try-power-role = "sink"; /* Prefer sink role */
                        op-sink-microwatt = <1000000>; /* Operating power required by the
sink, in microwatts */
                        orientation-switch = <&u2phy0>; /* Configure orien switch linked
nodes, supports up to 3 */
                        usb-role-switch = <&usb_drd0_dwc3>; /* Configure role switch linked
node */

                        /* Configure actual supported sink and source capabilities */
                        sink-pdos =
                                <PDO_FIXED(5000, 3000, PDO_FIXED_DATA_SWAP |
PDO_FIXED_USB_COMM | PDO_FIXED_EXTPOW
ER | PDO_FIXED_DUAL_ROLE)
                                 PDO_FIXED(9000, 3000, PDO_FIXED_USB_COMM)
                                 /*PDO_FIXED(12000, 3000, PDO_FIXED_USB_COMM)*/>;
```

```
                        source-pdos =
                                <PDO_FIXED(5000, 3000, PDO_FIXED_DATA_SWAP |
PDO_FIXED_USB_COMM | PDO_FIXED_EXTPOW
ER | PDO_FIXED_DUAL_ROLE)>;
                };
        };
};

/* USB2.0 PHY0 parent node configures property "orientation-switch", used to register Type-
C orientation switch callback function and control Bvalid signal */
&u2phy0 {
        orientation-switch;
        status = "okay";
};

/* USB2.0 PHY0 child node configures property "rockchip,typec-vbus-det", indicating support
for hardware design where Type-C VBUSDET pin is always pulled high */
&u2phy0_otg {
        rockchip,typec-vbus-det;
        /* rockchip,sel-pipe-phystatus; Optional property, select GRF to control pipe
phystatus, replacing USB3.0 ComboPHY control */
        status = "okay";
};

/* Configure USB3.0 DRD0 Controller to support USB2.0 only mode */
&usb_drd0_dwc3 {
        dr_mode = "otg";
        maximum-speed = "high-speed"; /* Configure maximum support only HS */
        phys = <&u2phy0_otg>; /* Configure phys property, only reference USB2.0 PHY0 */
        phy-names = "usb2-phy";
        snps,dis_u2_susphy_quirk; /* Disable suspend USB2.0 PHY function */
        snps,usb2-lpm-disable; /* Disable Host mode USB2 LPM function */
        snps,usb2-gadget-lpm-disable; /* Disable Device mode USB2 LPM function */
        usb-role-switch; /* Enable registration of Type-C USB Role Switch callback function
for switching DFP/UFP */
        status = "okay";
};
```

3.5 Type-C USB 2.0 only DTS Configuration

Refer to the DTS configuration of the Type-C USB 2.0 interface in `arch/arm64/boot/dts/rockchip/rk3572-iotest-v10.dtsi`.

```
/* Configure USB3.0/SATA/PCIe Combo PHY0 */
&combphy0_psu {
        rockchip,dis-u3otg0-port; /* If SATA/PCIe uses Combo PHY0, this property needs to be
added */
        status = "okay"; /* If SATA/PCIe does not use Combo PHY0, change this to status =
"disabled" */
};
```

```
/* Enable USB2.0 PHY0 parent node, since TCPC chip is not supported, no additional
configuration properties are needed */
&u2phy0 {
        status = "okay";
};

/* Enable USB2.0 PHY0 child node */
&u2phy0_otg {
        rockchip,vbus-always-on; /* Optional property, used to disable USB2.0 PHY charging
detection function */
        rockchip,sel-pipe-phystatus;  /* Select GRF to control pipe phystatus, replacing
USB3.0 ComboPHY control */
        status = "okay";
};

/* Configure USB3.0 DRD0 Controller to support USB2.0 only mode */
&usb_drd0_dwc3 {
        dr_mode = "peripheral"; /* Configure Device only mode, Host mode not supported */
        /* extcon = <&u2phy0>; Optional property, used for dr_mode = "otg" */
        maximum-speed = "high-speed";  /* Configure maximum support only HS */
        phys = <&u2phy0_otg>; /* Configure phys property, only reference USB2.0 PHY0 */
        phy-names = "usb2-phy";
        snps,dis_u2_susphy_quirk; /* Disable suspend USB2.0 PHY function */
        snps,usb2-gadget-lpm-disable; /* Disable Device mode USB2 LPM function */
        snps,usb2-lpm-disable; /* Disable Host mode USB2 LPM function */
        status = "okay";
};
```

3.6 Type-A USB 3.0 DTS Configuration

Refer to the DTS configuration of the Type-A USB 3.0 interface in `arch/arm64/boot/dts/rockchip/rk3572-evb1-v10.dtsi`.

```
/* Enable USB3.0/SATA/PCIe Combo PHY1 */
&combphy1_psu {
        status = "okay";
};

/* Enable USB2.0 PHY1 parent node */
&u2phy1 {
        status = "okay";
};

/* Enable USB2.0 PHY1 child node */
&u2phy1_otg {
        phy-supply = <&vcc5v0_usb3_host>; /* Configure property for VBUS power control, note
that this is not the PHY power supply */
        status = "okay";
};

/* Configure USB3.0 DRD1 Controller */
```

```
&usb_drd1_dwc3 {
        dr_mode = "host"; /* Configure Host only mode, Device mode not supported */
        status = "okay";
};
```

3.7 Multi Type-A USB 3.0 DTS Configuration

Refer to the DTS configuration of the Multi Type-A USB 3.0 interface (2 x Type-A USB3.0 + 3 x Type-A USB2.0) in `arch/arm64/boot/dts/rockchip/rk3572-evb3-v10.dtsi`.

```
/* Configure Reset Pin of external USB2.0 HUB chip */
vcc3v3_hubreset: vcc3v3-hubreset {
        compatible = "regulator-fixed";
        regulator-name = "vcc3v3_hubreset";
        ......
};

/* Configure VBUS control node for USB3.0 DRD0 interface */
vcc5v0_usb3_drd: vcc5v0-usb3-drd {
        compatible = "regulator-fixed";
        regulator-name = "vcc5v0_usb3_drd";
        ......
};

/* Configure VBUS control node for USB3.0 DRD1 interface */
vcc5v0_usb3_host: vcc5v0-usb3-host {
        compatible = "regulator-fixed";
        regulator-name = "vcc5v0_usb3_host";
        ......
};

/* Enable USB3.0/SATA/PCIe Combo PHY0 */
&combphy0_psu {
        status = "okay";
};

/* Enable USB3.0/SATA/PCIe Combo PHY1 */
&combphy1_psu {
        status = "okay";
};

/* Enable USB2.0 PHY0 parent node */
&u2phy0 {
        status = "okay";
};

/* Configure USB2.0 PHY0 child node */
&u2phy0_otg {
        vbus-supply = <&vcc5v0_usb3_drd>; /* vbus-supply is used for DRD mode, supporting
dynamic VBUS on/off */
        status = "okay";
```

```
};

/* Enable USB2.0 PHY1 parent node */
&u2phy1 {
        status = "okay";
};

/* Configure USB2.0 PHY1 child node */
&u2phy1_otg {
        phy-supply = <&vcc5v0_usb3_host>; /* phy-supply is used for HOST mode, VBUS is always
on after system powers on */
        status = "okay";
};

/* Configure USB3.0 DRD0 Controller */
&usb_drd0_dwc3 {
        dr_mode = "otg";
        extcon = <&u2phy0>; /* Configure extcon property, used to receive USB2.0 PHY
notification to switch to Device mode */
        status = "okay";
};

/* Configure USB3.0 DRD1 Controller */
&usb_drd1_dwc3 {
        dr_mode = "host"; /* Configure Host only mode, Device mode not supported */
        status = "okay";
};
```

3.8 USB3.0 Combo PHY Unpowered DTS Configuration

When the USB 3.0/SATA0/PCIe0 interfaces corresponding to USB3.0 Combo PHY0/1 are not used, the corresponding external power supply is allowed to be unpowered, which can reduce system power consumption and simplify hardware circuit design, but requires modifying the corresponding DTS configuration to avoid system boot abnormalities or USB function abnormalities.

3.8.1 Combo PHY0 Unpowered DTS Configuration

```
/* Disable USB3.0/SATA/PCIe Combo PHY0 */
&combphy0_psu {
        status = "disabled";
};

/* Configure USB3.0 DRD0 Controller to support only USB2.0 only mode */
&usb_drd0_dwc3 {
        maximum-speed = "high-speed";  /* Configure maximum support only HS */
        phys = <&u2phy0_otg>; /* Configure phys property, only reference USB2.0 PHY0 */
        phy-names = "usb2-phy";
        snps,dis_u2_susphy_quirk; /* Disable suspend USB2.0 PHY function */
        snps,usb2-gadget-lpm-disable; /* Disable Device mode USB2 LPM function */
```

```
        snps,usb2-lpm-disable; /* Disable Host mode USB2 LPM function */
        status = "okay";
};
```

It should be noted that the flashing tool must disable the USB 3.0 download function, otherwise the problem of Maskrom firmware download failure will occur. For the modification method, refer to the section Rockusb2.0 Download Tool Configuration.

3.8.2 Combo PHY1 Unpowered DTS Configuration

```
/* Disable USB3.0/SATA/PCIe Combo PHY1 */
&combphy1_psu {
        status = "disabled";
};

/* Configure USB3.0 DRD1 Controller to support only USB2.0 only mode */
&usb_drd1_dwc3 {
        maximum-speed = "high-speed";  /* Configure maximum support only HS */
        phys = <&u2phy1_otg>; /* Configure phys property, only reference USB2.0 PHY1 */
        phy-names = "usb2-phy";
        snps,dis_u2_susphy_quirk; /* Disable suspend USB2.0 PHY function */
        snps,usb2-gadget-lpm-disable; /* Disable Device mode USB2 LPM function */
        snps,usb2-lpm-disable; /* Disable Host mode USB2 LPM function */
        status = "okay";
};
```

3.9 USB DTS Important Property Configuration Instructions

3.9.1 USB Controller DTS Property Configuration Instructions

1. The property `usb-role-switch` is used for standard Type-C interfaces (with PD controller chip), and `dr_mode = "otg"` must be configured. If `dr_mode` is not `"otg"` mode, do not configure.
2. The property `extcon` is used to dynamically switch DRD mode, suitable for hardware circuit designs of non-standard Type-C interfaces, such as USB Micro interface or Type-A interface, and in schemes where the controller is configured as `dr_mode = "otg"`, to achieve dynamic switching between Device/Host mode.
3. The property `snps,usb2-lpm-disable` is used for disabling DWC3 Host mode USB2 LPM function

   ```
   &usb_drd0_dwc3 {
           snps,usb2-lpm-disable;
   };

   &usb_drd1_dwc3 {
           snps,usb2-lpm-disable;
   };
   ```

4. The property `snps,usb2-gadget-lpm-disable` is used for disabling DWC3 Device mode USB2 LPM function

```
&usb_drd0_dwc3 {
        snps,usb2-gadget-lpm-disable;
};
```

5. The property `snps,dis_u2_susphy_quirk` and `snps,dis_u3_susphy_quirk` is used for disabling DWC3 suspend USB2.0/3.0 PHY function

```
&usb_drd0_dwc3 {
        snps,dis_u2_susphy_quirk;
        snps,dis_u3_susphy_quirk;
};

&usb_drd1_dwc3 {
        snps,dis_u2_susphy_quirk;
        snps,dis_u3_susphy_quirk;
};
```

3.9.2 USB 2.0 PHY DTS Property Configuration Instructions

1. Both the property `vbus-supply` and the property `phy-supply` are used to control VBUS output 5V, but they have usage differences. `vbus-supply` is used for DRD mode, supporting dynamic VBUS on/off. `phy-supply` is used for USB HOST port, keeping VBUS 5V always on after system powers on.
2. The property `rockchip,sel-pipe-phystatus` is used to configure the GRF USB control register, selecting GRF to control pipe phystatus, replacing the control of USB 3.0 Combo PHY. Mainly used for USB 2.0 only schemes.
3. The property `rockchip,typec-vbus-det` is used to support hardware design where Type-C VBUS_DET is always pulled high.
4. The property `rockchip,vbus-always-on` is used for disabling USB 2.0 PHY charging detection function

```
&u2phy0_otg {
        rockchip,vbus-always-on;
};
```

   Note: After disabling the charging detection function, it will also disable the dynamic entering suspend function, and disable notifying the DRD controller driver to switch to Device mode. If the Device mode function is abnormal after disabling the charging detection function, it is recommended to configure the controller's `dr_mode` property to `"peripheral"`.

5. The property `rockchip,dis-u2-susphy` is used to disable USB 2.0 PHY dynamic entering suspend mode function, mainly used for USB 2.0 only schemes, to keep USB2 PHY output clock to the USB controller.

```
&u2phy0_otg {
        rockchip,dis-u2-susphy;
};
```

3.9.3 USB 3.0 Combo PHY DTS Property Configuration Instructions

1. The property `rockchip,dis-u3otg0-port` is used for USB3.0/SATA/PCIe Combo PHY0. When PHY0 is used for PCIe or SATA interface, add this property to disable USB 3.0 function.
2. The property `rockchip,dis-u3otg1-port` is used for USB3.0/SATA/PCIe Combo PHY1. When PHY1 is used for PCIe or SATA interface, add this property to disable USB 3.0 function.

3.9.4 Android GKI Dependency Property Configuration

There is a compatibility issue between the Android GKI USB controller driver and the USB 2.0 PHY dynamic entering and exiting suspend mode mechanism. Therefore, it is necessary to disable the USB 2.0 PHY dynamic entering suspend mode function. Refer to `arch/arm64/boot/dts/rockchip/rk3572-android.dtsi`

```
&u2phy0_otg {
        rockchip,dis-u2-susphy;
};
```

4. RK3572 USB DRD Mode Switching Commands

The RK3572 SDK supports forcibly setting the USB DRD controller to switch to Host or Device mode through software methods, without being affected by the DRD ID level of the USB hardware circuit or Type-C CC.

The RK3572 kernel supports two USB DRD mode switching methods:

Method 1.

```
#1.Force host mode
  echo host > /sys/devices/platform/soc/26054000.syscon/26054000.syscon:usb2-phy@0/otg_mode
#2.Force peripheral mode
  echo peripheral > /sys/devices/platform/soc/26054000.syscon/26054000.syscon:usb2-
phy@0/otg_mode
```

Method 2.

```
#1.Force host mode
  echo host > /sys/kernel/debug/usb/24000000.usb/mode
#2.Force peripheral mode
  echo device > /sys/kernel/debug/usb/24000000.usb/mode
```

Note:

Method 1 depends on the correct configuration of USB DTS and can only be used for hardware circuit designs of non-Type-C interfaces; Method 2 has no USB DTS configuration restrictions. Therefore, when it is uncertain whether the software and hardware are correctly adapted, it is recommended to use Method 2.

5. RK3572 USB Firmware Download Function Configuration

5.1 Rockusb3.0 Download Tool Configuration

5.1.1 Windows GUI Tool Configuration

Version requirements: RKDevTool V3.37 or higher, SocToolKit V2.5 or higher, FactoryTool V1.9.1 or higher.

Driver requirements: V5.13 or newer driver version. If the driver version is lower than V5.13, you can use the Rockchip Driver Assistant DriverAssitant_v5.13 to install the driver.

Configuration requirements: Configure the config.ini file of the tool to enable USB3 transfer function and configure the timeout for waiting for USB connection. According to different Windows tools, the modification references are as follows:

1. FactoryTool

   Configure config.ini (place the file in the same directory as FactoryTool.exe)

   ```
   #When USB3_TRANSFER=TRUE, if the device supports USB3 transfers, the tool will use USB3
   mode for downloading
   USB3_TRANSFER=TRUE
   #Modify the timeout to 5 seconds to reduce the time for falling back to USB2 after USB 3
   enumeration failure in MASKROM mode
   ROCKUSB_TIMEOUT=5
   ```

2. SocToolKit

   Configure config.ini (place the file in the bin\windows directory of the tool)

   ```
   usb3_transfer_on=true
   rkusb_timeout=5
   ```

3. RKDevTool

   Configure config.ini (place the file in the same directory as RKDevTool.exe)

   ```
   #When USB3_TRANSFER=TRUE, if the device supports USB3 transfers, the tool will use USB3
   mode for downloading
   USB3_TRANSFER=TRUE
   #Modify the timeout to 5 seconds to reduce the time for falling back to USB2 after USB 3
   enumeration failure in MASKROM mode
   ROCKUSB_TIMEOUT=5
   ```

5.1.2 Linux Command Line Tool Configuration

Version requirements: upgrade_tool V2.49 or higher.

Driver requirements: None.

Configuration requirements: Configure config.ini (place the file in the same directory as upgrade_tool)

```
usb3_transfer_on=true
rkusb_timeout=5
```

5.1.3 Mac Command Line Tool Configuration

Version requirements: upgrade_tool V2.44 or higher.

Driver requirements: None.

Configuration requirements: Configure config.ini (place the file in the same directory as upgrade_tool)

```
usb3_transfer_on=true
rkusb_timeout=5
```

5.2 Rockusb2.0 Download Tool Configuration

If the product hardware circuit design does not support USB 3.0 download function, the flashing tool must disable the USB 3.0 download function, otherwise the problem of Maskrom firmware download failure will occur. The modification method is to set the configuration parameter `USB3_TRANSFER` or `usb3_transfer_on` of the tool in the section Rockusb3.0 Download Tool Configuration to `false`.

5.3 Rockusb2.0 Kernel/U-Boot Configuration

1. Kernel DTS Configuration

   For circuit design schemes where Combo PHY0 is unpowered or unused

   ```
   /* Disable USB3.0/SATA/PCIe Combo PHY0 */
   &combphy0_psu {
           status = "disabled";
   };
   ```

   For circuit design schemes where Combo PHY0 is used for PCIe/SATA

   ```
   /* Configure USB3.0/SATA/PCIe Combo PHY0 */
   &combphy0_psu {
           rockchip,dis-u3otg0-port;
           status = "okay";
   };
   ```

2. U-Boot Configuration

Add `CONFIG_ROCKUSB_MAX_SPEED_HS=y` in the U-Boot project's `configs/rk3572-usbplug_defconfig` and `configs/rk3572_defconfig`

6. RK3572 USB 2.0 Low-Power Wakeup System Function Configuration

6.1 RK3572 USB 2.0 Wakeup Function Description

The scenarios related to USB 2.0 wakeup are as follows:

- Device mode: In standby state, detect host connection through the rising edge of bvalid to wake up the system.
- Host mode: In standby state, detect peripheral connection or Remote wakeup through DP/DM linestate level change interrupt to wake up the system.
- OTG mode: In standby state, detect OTG working mode switching through iddig level change interrupt to wake up the system.

These wakeup methods all require **USB 2.0 PHY to be in a powered state** to detect signal changes on DP/DM/VBUS/ID pins and generate interrupts, which are sent to the PMU to wake up the system.

Note. The interrupt detection of DP/DM linestate level change is different from the standard USB2 Remote wakeup mechanism.

The similarity is that both require the USB controller on the host side to support USB Remote Wakeup. The USB driver enables or disables the Remote Wakeup function of the USB peripheral through SET_FEATURE and CLEAR_FEATURE commands. Before the USB bus enters suspend, the Remote Wakeup feature should be enabled. When the USB peripheral wants to wake up the host, the peripheral will first initiate a Remote Wakeup signal lasting 3ms to notify the host system to start the USB Resume operation.

The difference is that the standard USB 2.0 Remote wakeup mechanism requires the host to send a response signal lasting more than 20ms after receiving the module's Remote Wakeup signal, and the host must start sending bus communication (SOF token) within 3ms of the Idle state, otherwise the device will enter Suspend again. Therefore, it requires the USB controller to maintain power supply during standby. The interrupt detection of DP/DM linestate level change does not require the host to send a Remote Wakeup response signal, so it allows the USB controller power supply to be turned off during standby.

6.2 RK3572 USB 2.0 Wakeup Function Hardware Circuit Design Requirements

Power supply requirements for secondary standby:

- The three power rails related to USB2 PHY: USB_DRD_DVDD0V75, USB_DRD_AVDD1V8, USB_DRD_AVDD3V3 must remain powered.
- USB VBUS needs to maintain external power supply.
- PMU maintains power supply.

- The OSC of the main controller can be powered off.
- The Logic of the main controller can be powered off.

Taking the hardware circuit reference design of the RK3572 EVB1 USB3.0 Host interface as an example:

1. The three power rails related to USB2 PHY are merged and connected to the PMU power supply VDD_0V75_S3/VCC_1V8_S3/VCC_3V3_S3

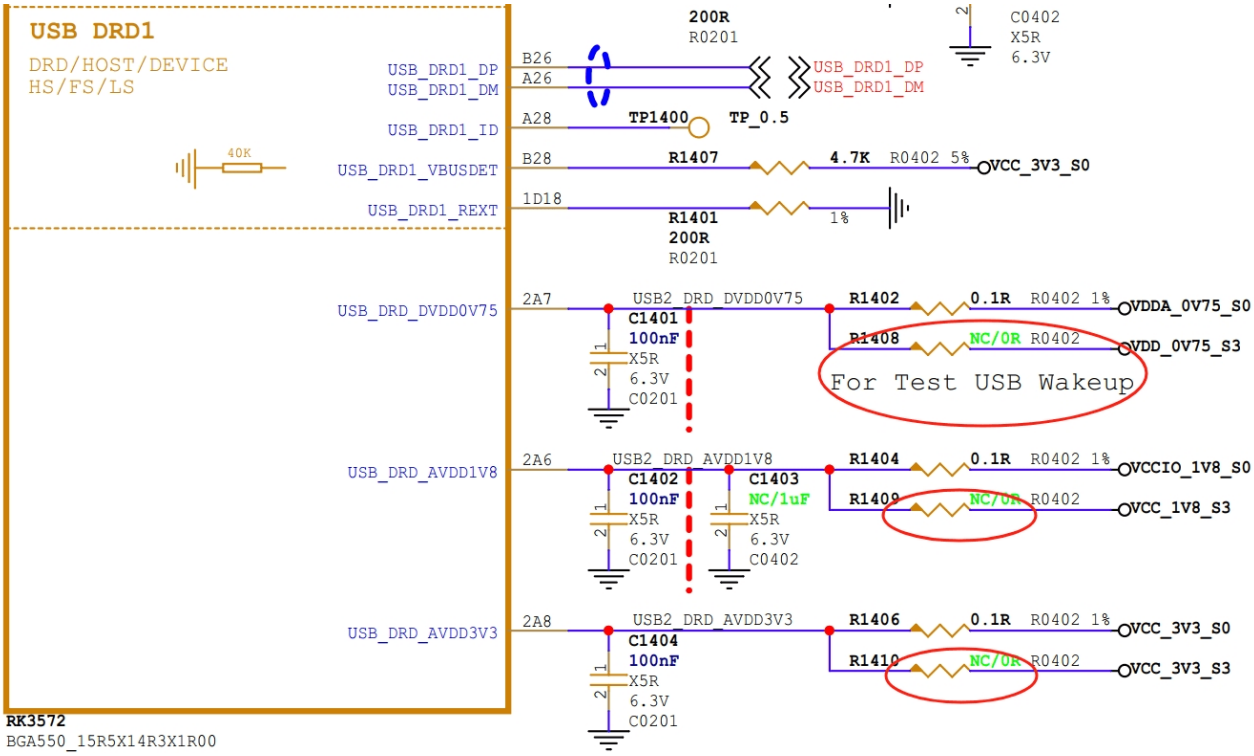


Figure 11 RK3572 USB 2.0 Wakeup Function PHY Power Supply Reference Design

2. The power supply for USB VBUS is modified from VCC5V0_DEVICE_S0 to VCC5v0_SYS_S5

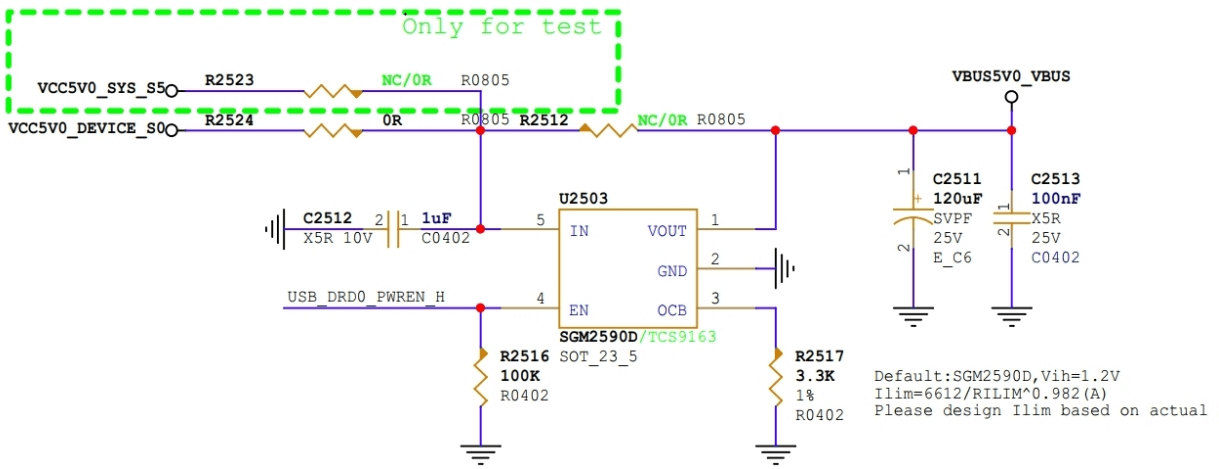


Figure 12 RK3572 USB 2.0 Wakeup Function VBUS Power Supply Reference Design

6.3 RK3572 USB 2.0 Wakeup Function Software Configuration

```
diff --git a/arch/arm64/boot/dts/rockchip/rk3572.dtsi
b/arch/arm64/boot/dts/rockchip/rk3572.dtsi
index 8555ba5d126a..2c8d394ffa71 100644
--- a/arch/arm64/boot/dts/rockchip/rk3572.dtsi
+++ b/arch/arm64/boot/dts/rockchip/rk3572.dtsi
@@ -1629,6 +1629,7 @@ rockchip_suspend: rockchip-suspend {
                rockchip,wakeup-config = <
                        (0
                        | RKPM_GPIO0_WKUP_EN
+                       | RKPM_USB_WKUP_EN
                        )
                >;
                rockchip,sleep-io-ret-config = <
```

7. RK3572 Type-C Controller Chip Support List

Table 8 RK3572 Type-C Controller Chip Support List

Chip	Linux-6.12	Description
AW35615	Supported	Recommended for priority use, good compatibility with DisplayPort Alt Mode Software and hardware fully compatible with FUSB302
ET7301B	Supported	Software and hardware fully compatible with FUSB302[note1]
ET7303	Supported	Hardware compatible with FUSB302, software driver highly similar to RT1711[note2]
FUSB302	Supported	Commonly used chip on RK platforms[note2]
HUSB311	Supported	Default chip used by RK3572 EVB1/2 Hardware compatible with FUSB302, but software driver not compatible[note3]
RT1711H	Supported	Hardware compatible with FUSB302, software driver highly similar to ET7303[note4]
RK809C	Supported	Default chip used by RK3572 Tablet[note5]

For more detailed information on the Type-C controller chip support list, please refer to the Redmine platform TYPE-C and PD support list:

https://redmine.rock-chips.com/documents/131

note1.

- Linux-6.12 adopts the TCPM software framework and TCPCI protocol, theoretically compatible with all Type-C controller chips designed based on the TCPCI standard (e.g., ET7303/HUSB311/RT1711H);

note2.

- Small package ET7303 (it is understood that the original manufacturer does not provide large package) has been verified on the RK platform. The kernel needs to separately enable CONFIG_TYPEC_ET7303.
- For detailed DTS configuration, please refer to the usbc0 node in the section Type-C USB 3.0 Switch Full-Function DTS Configuration. Only the node name, reg address, and compatible property need to be modified.

note3.

- FUSB302 can be directly replaced with HUSB311. The kernel needs to use CONFIG_TYPEC_HUSB311 separately. DTS configuration notes are the same as note2 above.

note4.

- RT1711H is hardware compatible with FUSB302/ET7303, while the software driver is highly similar to ET7303. The kernel needs to separately enable CONFIG_TYPEC_RT1711H. DTS configuration notes are the same as note2 above.

note5.

- RK809C is a PMIC chip that supports PD function, supporting up to 12V 3A charging. DTS configuration refers to the Tablet DEMO scheme Type-C USB 2.0 with PD DTS Configuration.

8. Reference Documents

1. Universal Serial Bus Type-C Cable and Connector Specification
2. Universal Serial Bus Power Delivery Specification
3. Rockchip_Developer_Guide_USB_EN
4. Rockchip RK3572 Technical Reference Manual